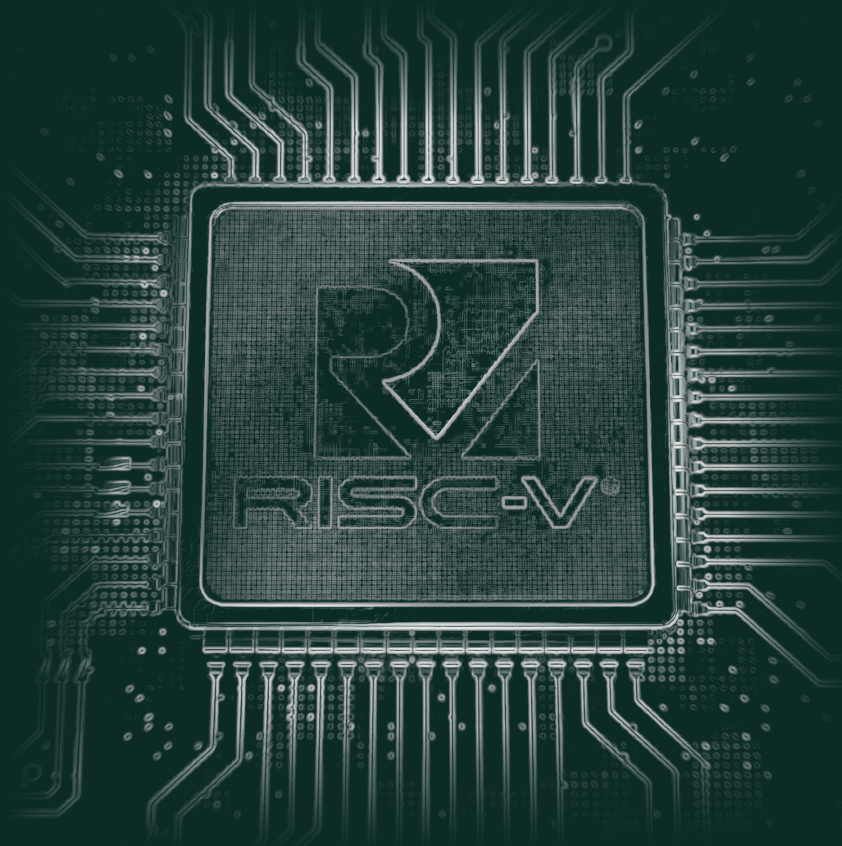




Universidad
de Jaén

Fundamentos de Arquitectura de Computadores

Programación en ensamblador RISC-V



Fundamentos de Arquitectura de Computadores

Programación en ensamblador RISC-V. Prácticas con Ripes

José L. López Ruiz

<llopez@ujaen.es>

José Santamaría López

<jslopez@ujaen.es>

Versión del 23 de septiembre de 2026

Sobre este documento

Este documento reúne, en un único material de consulta, los contenidos teóricos que se trabajan en las prácticas de la asignatura. Cada capítulo se corresponde con una de las prácticas:

Capítulo	Práctica
0. Conceptos previos y primer contacto con Ripes	Práctica 0
1. Introducción a RISC-V y Ripes	Práctica 1
2. Bucles y funciones en RISC-V	Práctica 2
3. Operaciones de entrada y salida	Práctica 3

Durante las sesiones de laboratorio se utiliza el simulador **Ripes** configurado con un procesador de **ciclo único** (single-cycle) y la ISA **RV32IM**. La instalación del simulador se describe en el material de la Práctica 0, y el recorrido por su interfaz, en el Capítulo 0.

El contenido teórico de este documento procede del libro *Programación en ensamblador RISC-V* de Francisco Charte Ojeda [1], empleado en la asignatura de continuación *Arquitectura de Computadores*. Aquí se recogen únicamente los capítulos que corresponden a esta asignatura. Como material complementario pueden consultarse el repositorio oficial del simulador Ripes [3] y el libro *Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition)* [2].

Índice

Sobre este documento	1
0 Conceptos previos y primer contacto con Ripes	5
0.1 Elementos básicos de la arquitectura de un computador	5
0.1.1 Las etapas de ejecución de una instrucción	6
0.1.2 El procesador de ciclo único (<i>single-cycle</i>)	7
0.2 La representación de la información	7
0.2.1 Bits, bytes y palabras	7
0.2.2 Binario, decimal y hexadecimal	7
0.2.3 Números con signo: el complemento a 2	8
0.2.4 Cómo se guardan los valores en la memoria	8
0.3 El simulador Ripes: primer contacto	9
0.3.1 La barra lateral: las vistas de Ripes	9
0.3.2 La barra de herramientas	10
0.3.3 La vista Editor	10
0.3.4 La vista Processor	11
0.3.5 La vista Memory	12
0.3.6 La vista I/O	13
0.3.7 Configurar el procesador de ciclo único	14
0.3.8 Guardar y cargar programas	15
1 Introducción a RISC-V y Ripes	16
1.1 La ISA de RISC-V	16
1.1.1 Banco de registros	16
1.1.2 Conjunto de instrucciones	17
1.1.3 Seudoinstrucciones	17
1.2 Primeros programas en Ripes	18
1.2.1 El editor de registros	18
1.2.2 El editor de código: primer programa	18
1.2.3 La consola y las llamadas al sistema	19
1.3 Instrucciones aritméticas	20
1.4 Carga y almacenamiento de datos	20
1.4.1 Espacio de direccionamiento y mapa de memoria	20
1.4.2 Leer y escribir datos en memoria	21
1.4.3 Examinar la memoria en Ripes	22
1.4.4 Un nuevo tipo de dato: el texto	23

1.5	Modos de direccionamiento	25
1.5.1	Los cuatro modos de direccionamiento de RISC-V	25
1.5.2	Formato general de las instrucciones RV32I	25
1.5.3	Instrucciones tipo R	26
1.5.4	Instrucciones tipo I y U	27
1.5.5	Instrucciones tipo S	29
1.6	Aspectos avanzados	30
1.6.1	Asignación de una dirección a un registro	30
1.7	Tomar decisiones: los saltos condicionales	32
1.7.1	La comparación, bit a bit	32
1.7.2	Un condicional en ensamblador	33
1.7.3	El salto por dentro: una suma al contador de programa	33
2	Bucles y funciones en RISC-V	36
2.1	Introducción	36
2.2	Instrucciones de salto	36
2.2.1	De los condicionales a los bucles	36
2.2.2	Saltos incondicionales	37
2.2.3	Llamadas a subrutinas	37
2.3	Otras instrucciones de la arquitectura RV32IM	39
2.3.1	Instrucciones lógicas	39
2.3.2	Instrucciones de comparación	40
2.3.3	Otras pseudoinstrucciones	41
2.4	Aspectos avanzados: la pila y las funciones	41
2.4.1	La pila como estructura de datos	41
2.4.2	Almacenamiento temporal de datos en la pila	42
2.4.3	Una función que llama a otra dentro de un bucle	45
2.4.4	Guardar datos de forma provisional: invertir una lista	46
3	Operaciones de entrada y salida	48
3.1	Introducción	48
3.2	Dispositivos de E/S en Ripes	48
3.2.1	Configuración de un dispositivo	49
3.2.2	Eliminación de dispositivos	50
3.3	Entrada/salida mapeada	50
3.3.1	Matriz de leds	51
3.3.2	Panel de microinterruptores	52
3.3.3	Conjunto de pulsadores	53
3.4	Otra entrada: recibir datos por la consola	54
3.5	E/S en hardware físico	55

3.5.1	E/S en microcontroladores	55
3.5.2	E/S en ordenadores	56
A	Repertorio de instrucciones RV32IM	58
	Referencias	62

Conceptos previos y primer contacto con Ripes

Antes de escribir la primera línea de ensamblador conviene asentar tres cosas. La primera, qué piezas forman un computador y cómo colaboran para ejecutar las instrucciones. La segunda, de qué manera representa la máquina la información con la que trabaja, desde un número sencillo hasta los negativos. Y la tercera, cómo manejarse con soltura por Ripes, el simulador que nos acompañará durante todo el curso. A ese triple objetivo dedicamos este capítulo, que se corresponde con la Práctica 0.

0.1 Elementos básicos de la arquitectura de un computador

Antes de empezar a programar en ensamblador conviene tener una imagen clara de los elementos que componen un computador y de cómo colaboran entre sí para ejecutar un programa. Todo computador (con independencia de su tecnología) se organiza en los llamados **cinco componentes clásicos** [2]: entrada, salida, memoria, camino de datos y control. Los dos últimos suelen combinarse y reciben el nombre de **procesador** o **CPU** (unidad central de proceso):

- Los **dispositivos de entrada y salida** comunican al computador con el mundo exterior; los trataremos en el Capítulo 3.
- La **memoria principal** es un conjunto de celdas numeradas (direccionables) en las que se almacenan tanto las instrucciones del programa como los datos sobre los que este opera: es el llamado *concepto de programa almacenado* (*stored-program concept*), base de todos los computadores modernos.
- El **camino de datos** es la parte del procesador que realiza las operaciones sobre los datos. En él encontramos:
 - La **unidad aritmético-lógica** (ALU), que realiza las operaciones de cálculo: sumas, restas, operaciones lógicas, comparaciones, etc.
 - El **banco de registros**, pequeñas porciones de memoria muy rápidas e internas a la CPU donde se alojan los datos con los que se está operando.
 - El **contador de programa** (registro **pc**), que contiene en todo momento la dirección de memoria de la instrucción a ejecutar.
- La **unidad de control** interpreta cada instrucción y envía las señales que determinan las operaciones del camino de datos, la memoria y la entrada/salida.

Esta organización clásica se conoce también como **arquitectura von Neumann**, en honor a John von Neumann, que ayudó a cristalizar estas ideas en los años cuarenta [2]. Las arquitecturas modernas (RISC-V incluida) se basan en esta arquitectura clásica: por distintos que sean los computadores actuales, sus piezas siguen encajando en estos cinco componentes.

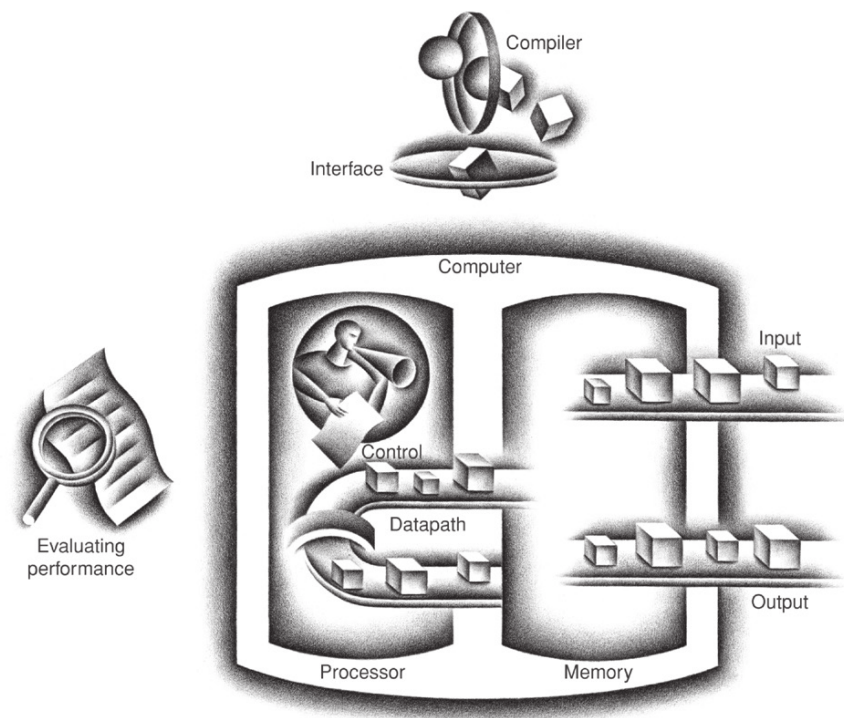


Figura 1: La organización de un computador, con sus cinco componentes clásicos: entrada (*Input*), salida (*Output*), memoria (*Memory*) y, dentro del procesador (*Processor*), el control (*Control*) y el camino de datos (*Datapath*). Figura tomada de [2].

La Figura 1 ilustra esta organización clásica: el procesador (con su control y su camino de datos) junto a la memoria y a los dispositivos de entrada y salida.

La CPU trabaja al ritmo que marca una señal periódica denominada **reloj**: cada pulso o **ciclo** de reloj marca el compás al que avanzan las operaciones internas del procesador. ¿Y qué ocurre exactamente dentro de esos ciclos? Para responder a esa pregunta hay que fijarse en el recorrido que sigue cada instrucción por el procesador.

0.1.1 Las etapas de ejecución de una instrucción

Todos los elementos anteriores entran en juego cada vez que se ejecuta una instrucción. En RISC-V, ese trabajo pasa clásicamente por cinco etapas [2]:

1. **Búsqueda** (*instruction fetch*, IF): se lee de memoria la instrucción cuya dirección indica el **pc**.
2. **Decodificación** (*instruction decode / register file read*, ID): la unidad de control interpreta la instrucción y se leen del banco de registros los operandos necesarios.
3. **Ejecución** (*execution*, EX): la ALU realiza la operación que corresponda o calcula una dirección de memoria.
4. **Acceso a memoria** (*memory access*, MEM): si la instrucción lo precisa (cargas y almacenamientos), se lee o escribe la posición de memoria correspondiente.
5. **Escritura de resultados** (*write-back*, WB): si procede, el resultado se escribe en el registro de destino.

Completadas las etapas, el **pc** se actualiza para pasar a la siguiente instrucción.

0.1.2 El procesador de ciclo único (*single-cycle*)

Conocidas las etapas, la siguiente pregunta es cómo se reparten en el tiempo, y aquí es donde cada implementación, es decir, cada procesador concreto, toma sus propias decisiones: una misma ISA puede ejecutarse de formas muy distintas. RISC-V es una arquitectura novedosa, concebida para trabajar con un **cauce segmentado** (*pipeline*): varias instrucciones avanzan por el procesador de manera simultánea, cada una en una etapa distinta de las anteriores. En este curso, sin embargo, trabajaremos siempre con una **ejecución secuencial**: hasta que una instrucción no termina, la siguiente no empieza.

El procesador que utilizaremos en Ripes es el de **ciclo único** (*single-cycle*). En este modo, las cinco etapas anteriores se completan dentro de un mismo ciclo de reloj, de forma que cada instrucción ocupa exactamente **un ciclo por instrucción**. Hay que tener presente, no obstante, que si cada etapa consumiera su propio ciclo de reloj, cada instrucción ocuparía realmente **5 ciclos**: el ciclo único es un ciclo «largo» que abarca todas las etapas de forma consecutiva.

0.2 La representación de la información

Ya sabemos qué hace cada pieza del computador. La siguiente pregunta es qué aspecto tiene, por dentro, la información que circula entre esas piezas. Los computadores almacenan y procesan la información como señales eléctricas altas y bajas, y por eso todo lo que manejan, ya sean números, texto o instrucciones, se representa en último término con solo dos símbolos: 0 y 1.

0.2.1 Bits, bytes y palabras

La unidad mínima de información es el **bit** (*binary digit*), que solo puede valer 0 o 1. Un grupo de 8 bits forma un **byte**, que es la unidad en la que se organiza la memoria: cada celda de memoria almacena un byte. En RV32, el tamaño natural de trabajo es la **palabra**, de 32 bits (4 bytes): es el ancho de los registros y de las instrucciones. Los bits de una palabra se numeran del 31 al 0: el bit 0, el de más a la derecha, es el **menos significativo** (el de menor peso), y el bit 31, el de más a la izquierda, el **más significativo**.

0.2.2 Binario, decimal y hexadecimal

En binario, igual que en decimal, cada posición tiene un peso: potencias de 2 en lugar de potencias de 10. Así, el número binario 1011 vale

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11.$$

Escribir valores de 32 bits en binario es incómodo y propenso a errores. Por eso se emplea la base 16 o **hexadecimal**: como $16 = 2^4$, cada dígito hexadecimal equivale exactamente a **4 bits**, y una palabra de 32 bits se escribe con solo 8 dígitos. Sus dígitos son 0–9 y A–F:

Hex	Binario	Hex	Binario	Hex	Binario	Hex	Binario
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

Los valores hexadecimales se escriben con el prefijo 0x. Convertir entre binario y hexadecimal es inmediato: basta traducir cada dígito hexadecimal a su grupo de 4 bits. Por ejemplo, para 0x2A3:

$$0x2A3 = \underbrace{0010}_2 \underbrace{1010}_A \underbrace{0011}_3$$

Su valor decimal se obtiene aplicando los pesos posicionales de la base 16, igual que antes hicimos en binario con los de la base 2 (recuerda que A = 10):

$$2 \times 16^2 + 10 \times 16^1 + 3 \times 16^0 = 512 + 160 + 3 = 675.$$

Las direcciones de memoria de 32 bits se escriben siempre en hexadecimal, con 8 dígitos: de 0x00000000 a 0xFFFFFFFF. Te encontrarás esta notación constantemente en Ripes.

0.2.3 Números con signo: el complemento a 2

¿Cómo se representan los números negativos si solo tenemos ceros y unos? El convenio empleado, llamado **complemento a 2**, reserva el bit más significativo como **bit de signo**: 0 para positivos, 1 para negativos. Para obtener el negativo de un número se **invierten todos sus bits y se suma 1**. Con 8 bits, por ejemplo:

$$5 = 0000\ 0101 \xrightarrow{\text{invertir}} 1111\ 1010 \xrightarrow{+1} 1111\ 1011 = -5.$$

La gracia del convenio es que la suma ordinaria funciona sin reglas especiales: $5 + (-5) = 0000\ 0101 + 1111\ 1011 = 1\ 0000\ 0000$, y descartando el acarreo que no cabe en 8 bits, el resultado es 0, como debe ser.

Con n bits, el rango representable es $[-2^{n-1}, 2^{n-1} - 1]$. Con los 32 bits de un registro: de $-2\ 147\ 483\ 648$ a $2\ 147\ 483\ 647$. Y con los 12 bits del valor inmediato de instrucciones como **addi**: de -2048 a 2047 , como veremos en el Capítulo 1.

Cuando un valor con signo se copia a un registro más ancho, no basta con rellenar con ceros: se replica el bit de signo en los bits nuevos, operación conocida como **extensión de signo**. Así, el -5 de 8 bits pasa a 16 bits como **1111 1111 1111 1011**. Instrucciones como **lb** o **lh**, que cargan datos más estrechos que la palabra, la aplican automáticamente.

Un mismo patrón de bits puede interpretarse como número con signo, sin signo o en hexadecimal. En Ripes, la lista desplegable *Display type* del panel de registros (*GPR*) y de la vista *Memory* (*Signed*, *Unsigned*, *Hex*) te permite ver las tres lecturas del mismo valor.

0.2.4 Cómo se guardan los valores en la memoria

Sabiendo ya qué aspecto tienen los números, veamos dónde se almacenan. La memoria es un gran conjunto de celdas de 1 byte, numeradas por su dirección. Una palabra de 32 bits ocupa **4 bytes consecutivos**, por lo que las direcciones de dos palabras consecutivas se diferencian en 4: 0x10000000, 0x10000004, 0x10000008...

Queda una última pregunta: dentro de esos 4 bytes, ¿cuál se guarda primero? RISC-V pertenece al grupo de arquitecturas **little-endian**: el byte **menos significativo** ocupa la dirección más baja. La palabra 0x12345678 almacenada en la dirección 0x10000000 queda así:

Dirección	0x10000000	0x10000001	0x10000002	0x10000003
Contenido	0x78	0x56	0x34	0x12

Puedes comprobarlo en la vista *Memory* de Ripes: declara un dato con `.word 0x12345678`, ejecuta el programa y examina la sección `.data` con *Display type* en *Hex*.

0.3 El simulador Ripes: primer contacto

Nos queda por conocer el taller en el que trabajaremos. Aunque hay disponibles distintos tipos de hardware con procesadores RISC-V, a través de un simulador resulta mucho más sencillo visualizar ciertos aspectos del funcionamiento interno del procesador. Por esa razón recurriremos a un software de simulación, que nos facilitará la edición y ejecución de programas, permitiendo examinar el contenido de los registros, memoria y algunos dispositivos de entrada/salida, algo difícil de hacer en hardware real. De entre los simuladores RISC-V disponibles elegiremos **Ripes** [3], un entorno de trabajo que aúna editor, ensamblador, visor del cauce de ejecución, visor de memoria, emulación de dispositivos de E/S, etc. La descarga e instalación se tratan en el material de la Práctica 0; aquí haremos el mismo recorrido por su interfaz, con algo más de detalle.

0.3.1 La barra lateral: las vistas de Ripes

Ripes se organiza en **vistas** que se eligen en la barra lateral izquierda, la de los iconos con fondo oscuro (Figura 2). De arriba a abajo:

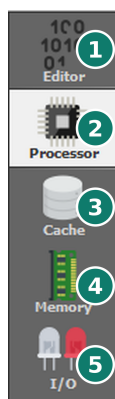


Figura 2: La barra lateral de Ripes, con los botones que cambian de vista numerados según el texto.

1. **Editor**: donde se escribe el código ensamblador.
2. **Processor**: el camino de datos del procesador y su ejecución.
3. **Cache**: la memoria caché (no se usa en esta asignatura).
4. **Memory**: el contenido y el mapa de la memoria.
5. **I/O**: los dispositivos de entrada/salida (Capítulo 3).

0.3.2 La barra de herramientas

Desde la barra superior (Figura 3) se controla la ejecución del programa. De izquierda a derecha:



Figura 3: La barra de herramientas de Ripes, con los controles numerados según el texto.

1. **Seleccionar el procesador:** abre el diálogo de configuración que veremos en la Sección 0.3.7.
2. **Reiniciar** la simulación.
3. **Retroceder** una instrucción (deshace el último ciclo).
4. **Ejecutar la siguiente instrucción** (paso a paso).
5. **Ejecutar el programa** de forma automática.
6. **Intervalo** entre ciclos de la ejecución automática, en milisegundos (cuanto menor, más rápida).
7. **Ejecutar hasta el final.**

En lugar de los botones podemos usar los atajos de teclado **F5** (ejecutar un ciclo), **F4** (deshacerlo) y **F8** (ejecutar hasta la finalización). La ejecución se restablece de manera automática si se modifica el código en el editor. A la derecha de estos controles, algunas vistas añaden un botón propio: en *Editor*, el de compilar un programa en C (inactivo cuando se trabaja en ensamblador), y en *Processor*, el que muestra el diagrama del cauce.

0.3.3 La vista Editor

La Figura 4 muestra la vista *Editor*, la que más usaremos. Sus elementos, según la numeración de la captura:

1. *Source code*: aquí se escribe el programa en ensamblador.
2. *Input type*: selecciona *Assembly* (o C).
3. *Executable code*: el código máquina que genera cada línea, en binario (*Binary*) o desensamblado (*Disassembled*).
4. *Registers* (GPR): el estado de los 32 registros; el que cambia durante la ejecución se resalta (en la captura, el puntero de pila **sp**, que Ripes inicializa a **0x7fffffff0**).

La parte inferior la ocupa la *Console*, donde el programa escribe su salida.

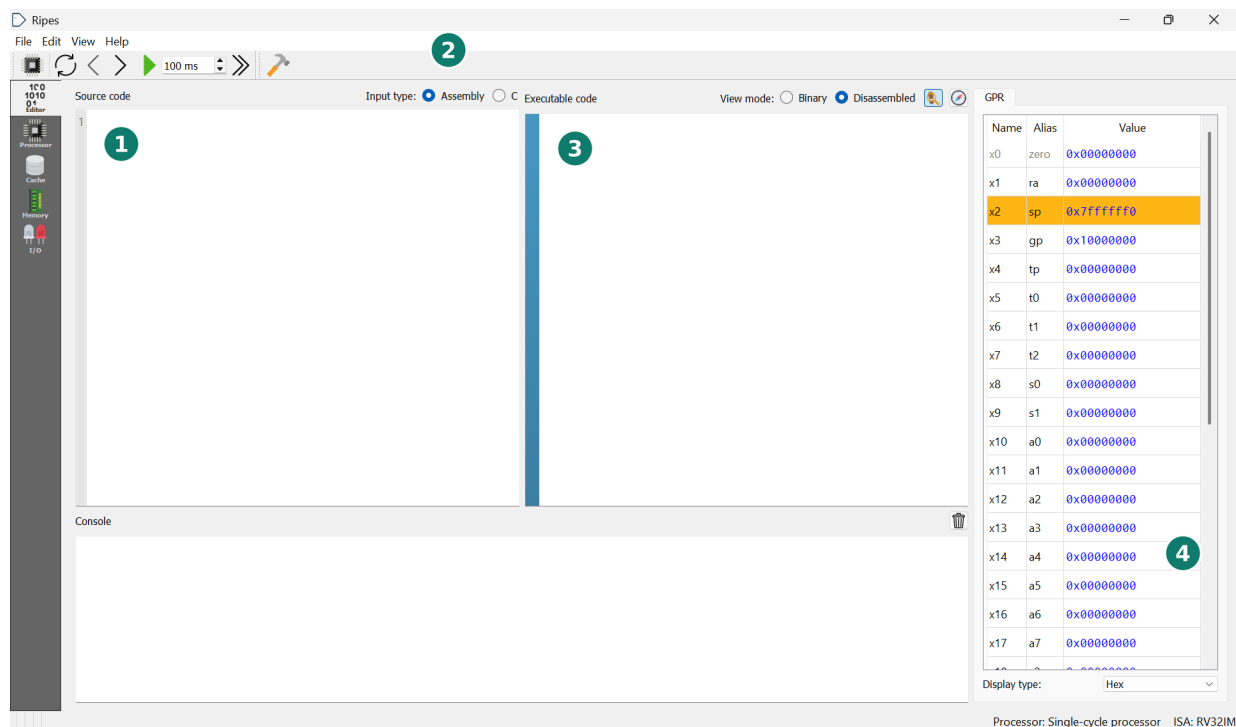


Figura 4: Vista *Editor*, con los elementos numerados según el texto.

0.3.4 La vista Processor

La vista *Processor* (Figura 5) muestra la ejecución por dentro. Sus elementos, según la numeración de la captura:

1. El **camino de datos** del procesador de ciclo único: el **pc**, la memoria de instrucciones, la decodificación, el banco de registros, la ALU y la memoria de datos, es decir, las piezas que presentamos en la Sección 0.1. La memoria aparece dibujada dos veces (memoria de instrucciones y memoria de datos), pero ambos bloques acceden a la misma memoria principal.
2. *Registers*: el valor de cada registro.
3. *Execution info*: ciclos ejecutados, instrucciones retiradas, CPI, IPC y frecuencia.
4. *Instruction memory*: las instrucciones cargadas y su dirección.
5. *Console*: mensajes y salida del programa.

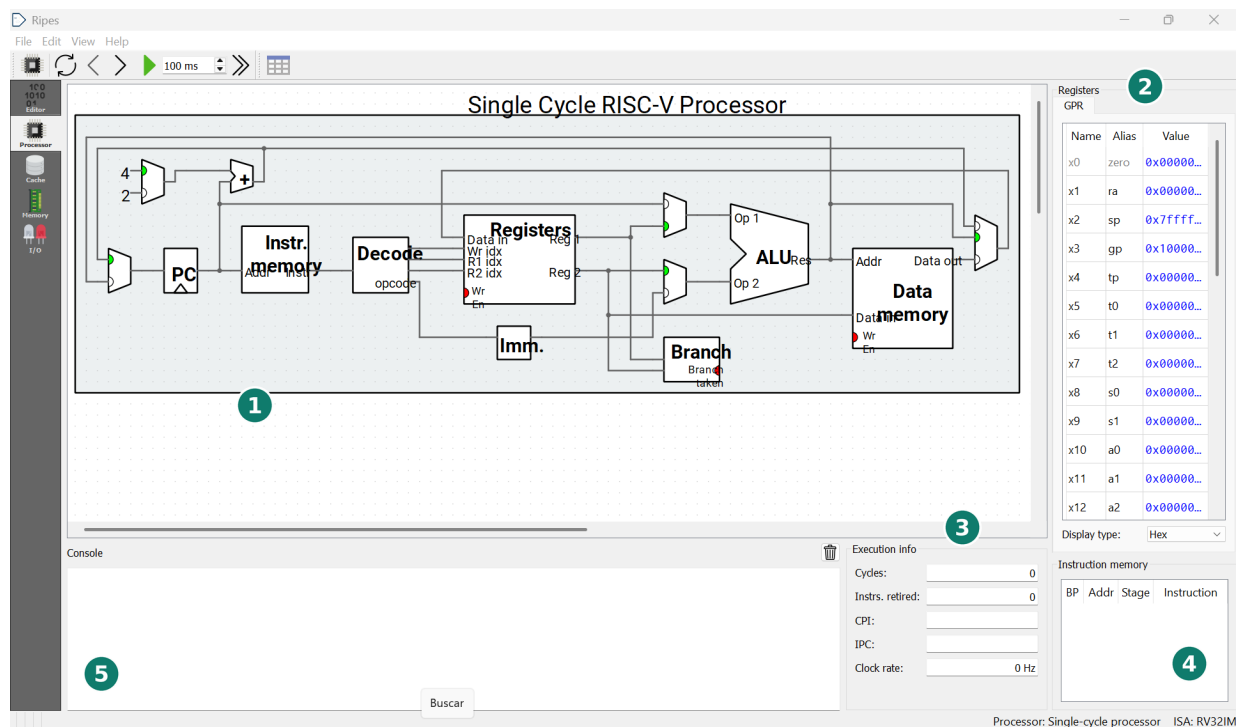


Figura 5: Vista *Processor*, con los elementos numerados según el texto.

0.3.5 La vista *Memory*

La vista *Memory* (Figura 6) permite examinar la memoria. Sus elementos, según la numeración de la captura:

1. *Memory viewer*: cada dirección y su contenido, como palabra completa (*Word*) y desglosado por bytes (*Byte 0-3*) (ahí puedes comprobar el orden *little-endian* que vimos en la Sección 0.2.4).
2. *Memory map*: las secciones `.text`, `.data` y `.bss` con sus rangos de direcciones.
3. *Go to section / Go to register*: salta directamente a una sección o a la dirección contenida en un registro.
4. *Display type*: el formato de visualización (hexadecimal, decimal...).

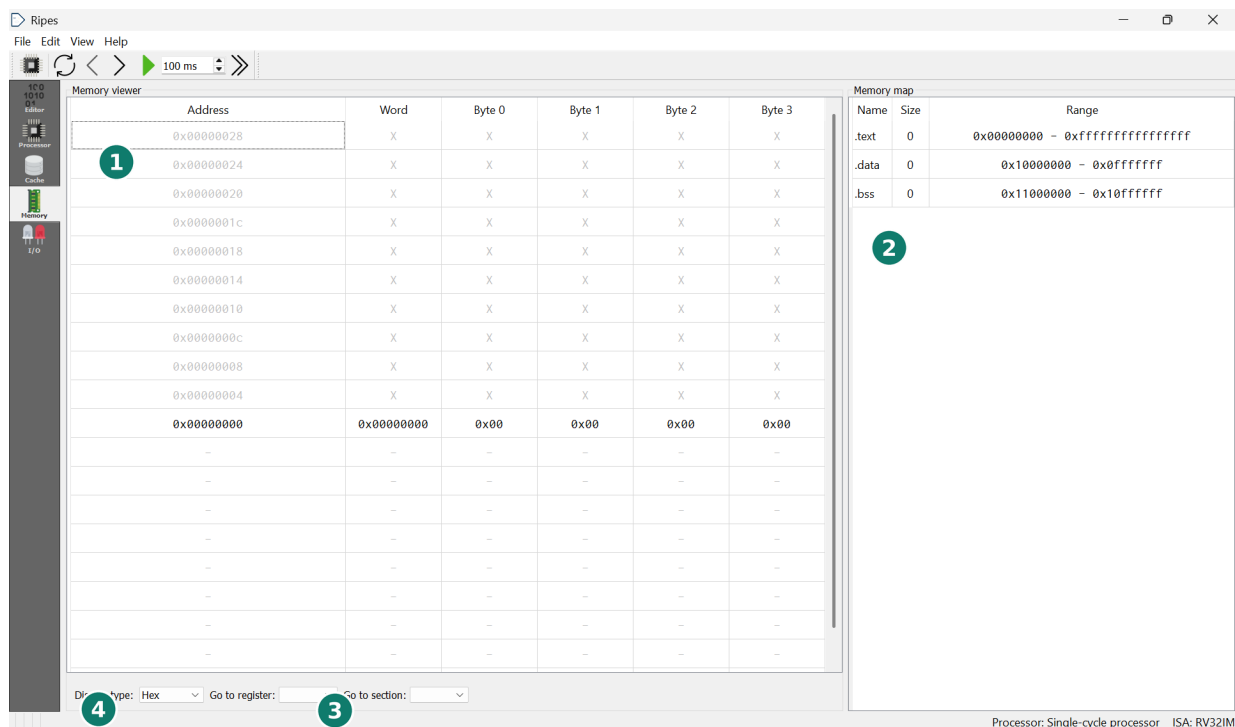


Figura 6: Vista *Memory*, con los elementos numerados según el texto.

0.3.6 La vista I/O

La vista *I/O* (Figura 7) gestiona los dispositivos de entrada/salida. Sus elementos, según la numeración de la captura:

1. *Devices*: la lista de dispositivos disponibles, a la izquierda.
2. Los dispositivos añadidos aparecen en el **área central**: matrices de leds (*LED Matrix*), microinterruptores (*Switches*) y pulsadores (*D-Pad*).
3. *Register map*: los registros del dispositivo seleccionado, con su dirección relativa a la dirección base (*Address*), si se leen o se escriben (*R/W?*) y su tamaño en bits (*Size*).
4. *Exports*: las constantes `#define` con la dirección base de cada dispositivo, el tamaño en bytes de su espacio de E/S y otros valores (p. ej. `SWITCHES_0_BASE` y `SWITCHES_0_SIZE`): es la E/S mapeada en memoria.

La usaremos a fondo en el Capítulo 3.

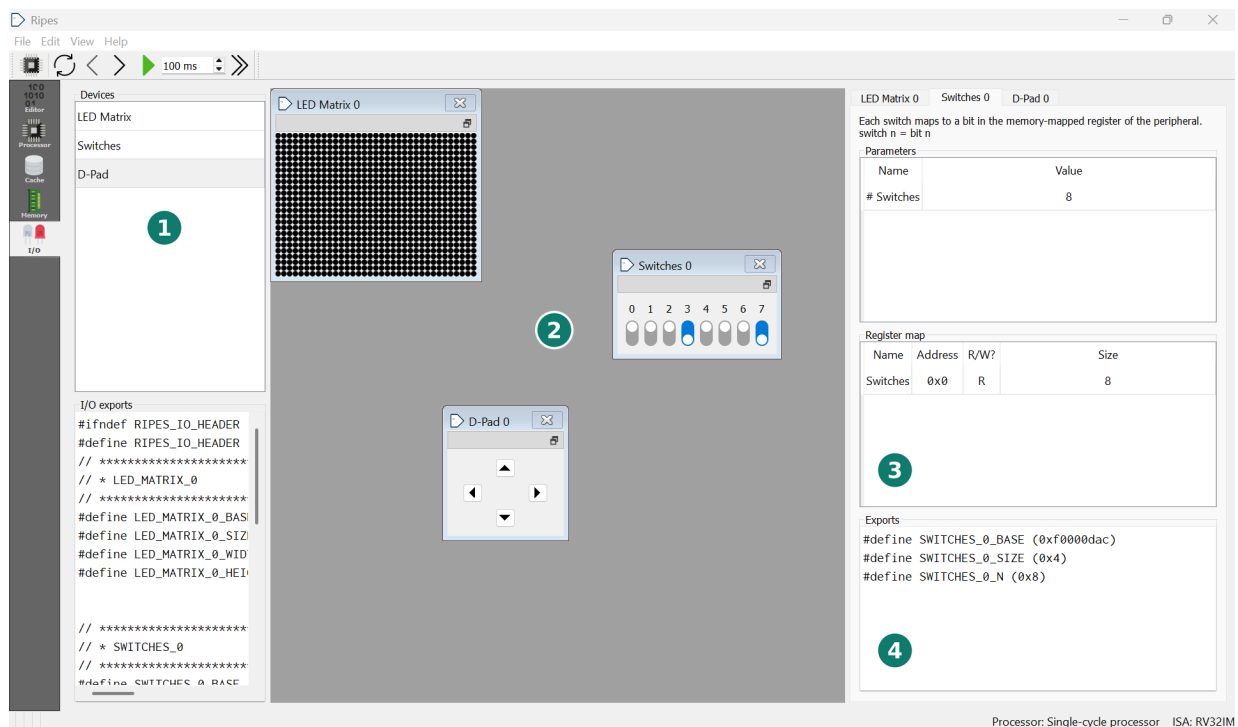


Figura 7: Vista I/O, con los elementos numerados según el texto.

0.3.7 Configurar el procesador de ciclo único

El primer botón de la barra de herramientas abre el diálogo *Select Processor* (Figura 8), donde se elige el procesador a simular. Nuestra configuración para todo el curso:

1. En el árbol de la izquierda, elige *RISC-V* → *32-bit* → **Single-cycle processor**: el procesador de ciclo único, en el que cada instrucción ocupa un ciclo.
2. Comprueba que la ISA es *RV32I* con la extensión **M** marcada: trabajamos con RV32IM.
3. *Layout: Standard*.
4. En *Register initialization*, el puntero de pila **x2 (sp)** se inicializa a `0x7fffffff0`, el tope inicial de la pila (lo situaremos en el mapa de memoria de la Figura 1.1).
5. Pulsa *OK*. La barra de estado inferior confirmará *Processor: Single-cycle processor* e *ISA: RV32IM*.

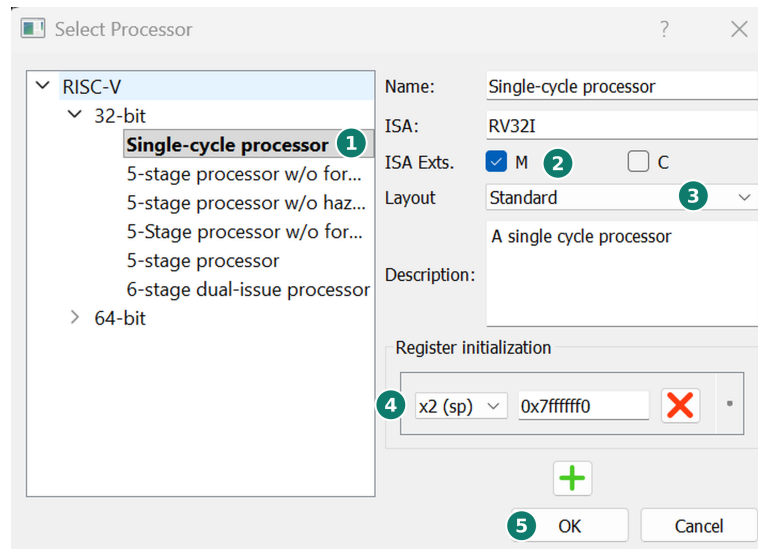


Figura 8: El diálogo *Select Processor* con la configuración de la asignatura, numerada según los pasos del texto.

0.3.8 Guardar y cargar programas

Las opciones para almacenar el código de un programa en disco (guardándolo en un archivo que habitualmente tendrá extensión `.s`) o bien para cargar un programa guardado con anterioridad, las encontramos en el menú *File*. Esta será la vía para guardar los ejercicios en archivos y entregarlos a través de la plataforma de docencia virtual en las sesiones de evaluación. En el mencionado menú también encontramos la opción *Load Example*→*Assembly*, mediante la cual es posible abrir diferentes programas de ejemplo provistos por el propio software.

Introducción a RISC-V y Ripes

Esta primera práctica es tu punto de entrada al mundo del lenguaje ensamblador, el lenguaje en el que se expresa directamente lo que hace el procesador. Las piezas que componen el computador y el manejo básico de Ripes los conocimos en el Capítulo 0; ahora presentaremos la arquitectura RISC-V y su ISA (el «idioma» que entiende el procesador), con su banco de registros y su conjunto de instrucciones, y escribiremos nuestros primeros programas: los ejecutaremos y examinaremos sus efectos sobre los registros y la memoria.

1.1 La ISA de RISC-V

En el Capítulo 0 hemos hablado del hardware: registros, ALU, memoria y el camino de datos que los conecta. Para programar ese hardware necesitamos conocer la interfaz que nos ofrece. Se denomina **ISA** (*Instruction Set Architecture*) a la interfaz que determina cómo mediante software se controla el hardware de un microprocesador. La ISA de RISC-V nos permitirá escribir programas para cualquier CPU que implemente dicha ISA. Forman parte de la ISA aspectos como los siguientes:

- El banco de registros visibles a los programas.
- Los modos de direccionamiento que facilitan el acceso a memoria.
- El conjunto de instrucciones que ofrece para realizar distintas operaciones.

En estas prácticas usaremos la ISA de RISC-V denominada **RV32IM**: *RISC-V 32 bits Integer and Multiplication*, en su versión 2.0. El ancho de palabra, y de las instrucciones, es de 32 bits y se contemplan instrucciones de aritmética entera, incluyendo la multiplicación y división.

1.1.1 Banco de registros

Los registros son pequeñas porciones de memoria internas a la CPU. Su finalidad es mantener temporalmente los datos con que se opera y los resultados que producen las operaciones sobre ellos. En RV32I contamos con **32 registros de propósito general**, cada uno con un tamaño de 32 bits. La denominación genérica para estos registros es **xN**, siendo N un número entre 0 y 31.

A pesar de la denominación de propósito general, algunos de esos 32 registros se emplean para fines específicos según una convención para el desarrollo de compiladores para RISC-V, la denominada **ABI** (*Application Binary Interface*). La Tabla 1.1 recoge esos usos estandarizados y los nombres alternativos (alias) con los que es posible acceder a los registros.

Además de estos, la arquitectura también cuenta con el registro **pc** (*Program Counter*) (que contiene la dirección de la instrucción a ejecutar, y es también un registro de 32 bits en RV32I) así como otros registros de uso específico que, por el momento, no nos resultan de interés.

Al escribir un programa en ensamblador somos libres de emplear el nombre original de cada registro o bien su alias, sin que esto afecte en forma alguna al resultado. Es habitual optar por la segunda opción, ya que los alias tienen un sentido semántico del que carecen los nombres reales.

Nombre	Alias	Uso
x0	zero	Siempre contiene el valor 0
x1	ra	Dirección de retorno
x2	sp	Puntero de pila
x3	gp	Puntero global
x4	tp	Puntero de hilo
x5 a x7	t0 a t2	Valor temporal
x8	s0/fp	Registro preservado / Puntero de <i>frame</i>
x9	s1	Registro preservado
x10, x11	a0, a1	Parámetro de función / Valor de retorno
x12 a x17	a2 a a7	Parámetro de función
x18 a x27	s2 a s11	Registro preservado
x28 a x31	t3 a t6	Valor temporal

Tabla 1.1: Conjunto de registros de propósito general en RISC-V. La denominación *registro preservado* significa que el contenido de ese registro no será modificado al realizar una llamada a una función, por lo que su valor se preserva.

1.1.2 Conjunto de instrucciones

Además de su banco de registros, para escribir programas necesitaremos conocer también las instrucciones con que cuenta RISC-V. Este conjunto cambia según las extensiones de la ISA que implemente el procesador. Para RV32I es bastante reducido, al constar de solo **40 instrucciones** distintas, con variantes según de dónde se tomen los operandos o el tipo de los mismos. A estas, RV32IM añade varias más para el cálculo de multiplicaciones y divisiones. La Tabla 1.2 resume las que usaremos (la extensión M incluye además *mulh*, *mulhsu*, *mulhu*, *divu* y *remu*).

Grupo	Registro	Inmediato
Aritmético	add, sub, mul, div, rem	addi, lui, auipc
Lógico	and, or, xor	andi, ori, xori
Desplazamiento	sll, srl, sra	slli, srli, srai
Comparación	slt, sltu	slti, sltiu
Carga		lb, lbu, lh, lhu, lw
Almacenamiento		sb, sh, sw
Salto		beq, bne, bge, bgeu, blt, bltu, jal, jalr
Otras	ecall, ebreak	

Tabla 1.2: Conjunto de instrucciones de la ISA RV32IM, dividido en grupos y según que ambos operandos estén en registros o uno de ellos sea un valor inmediato.

1.1.3 Seudoinstrucciones

La filosofía de los procesadores RISC, en términos generales, se basa en una ISA con pocas instrucciones encargadas de facilitar operaciones básicas. Esto hace que la implementación de dicha ISA en un microprocesador sea relativamente sencilla. El inconveniente es que se precisan más instrucciones ensamblador para realizar ciertas tareas, mientras que en un diseño CISC existirían instrucciones complejas para llevarlas a cabo.

RISC-V contempla el uso de **seudoinstrucciones** que, desde la perspectiva del programador, no son más que instrucciones algo más complejas. Esas pseudoinstrucciones son procesadas por el

ensamblador (el programa que traduce el código ensamblador a código máquina), no por el propio microprocesador, de forma que cada una de ellas genera una o múltiples instrucciones nativas. Será la herramienta con la que trabajemos, por ejemplo el simulador Ripes, la que se encargue de traducirlas y producir las instrucciones que correspondan.

1.2 Primeros programas en Ripes

Conocido ya el entorno de trabajo (Capítulo 0), es el momento de empezar a programar en él.

1.2.1 El editor de registros

En las vistas *Processor* y *Editor* tendremos en el margen derecho el editor de registros, con el nombre **GPR** (*General Purpose Registers*) en la parte superior. Se trata de una lista con el nombre, alias y valor actual de cada uno de los registros de propósito general. Sobre este editor es posible llevar a cabo distintas acciones:

- **Ayuda:** se obtiene una descripción de cualquier registro al situar el puntero del ratón sobre su nombre.
- **Formato:** con la lista desplegable situada en la parte inferior se elige el formato de visualización.
- **Notificación:** durante la ejecución de un programa, se notifica la escritura en cualquier registro cambiando el color de fondo con el que aparece en el editor.
- **Modificación:** con un doble clic sobre el valor se permite cambiar su contenido.

El registro `sp` actúa como puntero de pila. RISC-V no cuenta con un registro específico para esta función, como ocurre con otras arquitecturas de procesador, por lo que se emplea uno de los GPR para ello. Ripes lo inicializa con un valor adecuado al arrancar.

1.2.2 El editor de código: primer programa

Tras cambiar a la vista del *Editor* de código estamos en disposición de introducir órdenes en ensamblador RISC-V, en las que emplearemos una instrucción (de las enumeradas en la Tabla 1.2) y uno o más registros (de los recogidos en la Tabla 1.1). Comencemos por algo muy sencillo: alojar dos valores en sendos registros y sumarlos, mostrando el resultado en la consola de Ripes.

```
1  # a1, a2: sumandos
2  # a0:      resultado de la suma
3  # a7:      codigo de servicio para ecall
4  addi a1, zero, 3      # a1 = 3
5  addi a2, zero, 7      # a2 = 7
6  add a0, a1, a2        # a0 = a1 + a2
7  addi a7, zero, 1
8  ecall                # Llamar a funcion 1 (imprimir entero en a0)
9  addi a7, zero, 10
10 ecall                 # Llamar a la funcion 10 (exit)
```

Listado 1.1: Cargar dos valores en sendos registros y sumarlos.

A medida que introducimos estas instrucciones en el editor, en el panel de la derecha aparecerá el código máquina que generan (en binario) o bien su versión desensamblada, según la opción que esté marcada en la parte superior.

Como es habitual en procesadores RISC, no existe una instrucción para cargar en un registro un valor inmediato, de ahí que se emplee la instrucción `addi` con el registro `zero` y el valor que interese. Este uso es tan común que existe una seudoinstrucción (`li rd, imm`) que genera la anterior instrucción. Tampoco se contempla la copia de un registro a otro, para lo cual se puede emplear la seudoinstrucción `mv`.

1.2.3 La consola y las llamadas al sistema

La parte inferior de la vista *Editor* está ocupada por un panel con el título **Console**, cuya finalidad es mostrar información enviada por el programa a la salida estándar. Pero, ¿cómo escribe ahí un programa? Nuestras instrucciones saben operar con registros y con memoria, y nada más: para todo lo que quede fuera (escribir en la consola, terminar la ejecución) hay que pedirselo al entorno.

Ese es exactamente el trabajo de la instrucción `ecall` (*environment call*, llamada al entorno). Al ejecutarla, el programa cede el control al entorno que lo ejecuta, en nuestro caso el simulador Ripes, para solicitarle un **servicio**; atendida la petición, la ejecución continúa por la instrucción siguiente. La comunicación sigue una convención de registros fija:

- **a7** lleva el **código del servicio**: qué se pide. Cada servicio está identificado con un código numérico.
- **a0** (y, si el servicio necesita más datos, **a1**, **a2**...) lleva los **argumentos** de entrada.
- Si el servicio produce un resultado, se **devuelve** también en **a0**. Hay servicios que devuelven resultados y otros que no.

Estos son los servicios de Ripes que usaremos en las prácticas:

a7	Servicio	Argumentos	Efecto
1	Imprimir entero	a0 = entero	Escribe a0 en decimal en la consola
4	Imprimir cadena	a0 = dirección	Escribe la cadena hasta el terminador nulo
10	Terminar	(ninguno)	Pone fin a la ejecución del programa
34	Imprimir en hex	a0 = entero	Escribe a0 en hexadecimal
35	Imprimir en binario	a0 = entero	Escribe a0 en binario, con 32 dígitos

Así, la pareja `li a7, 1 + ecall` envía a la consola el entero que haya en **a0**, y `li a7, 10 + ecall` termina el programa. Es lo que hicimos en el Listado 1.1, con su equivalente `addi a7, zero, 1` (y `addi a7, zero, 10`).

Los códigos de servicio a los que se invoca con `ecall` dependen del sistema en que se ejecute el programa. Los indicados aquí son los del simulador Ripes: usa la opción *Help→System calls* para abrir una ventana en la que se detallan los códigos de servicio implementados y sus parámetros de entrada y salida.

1.3 Instrucciones aritméticas

Nuestro primer programa se limitaba a sumar. Además de la operación de suma, RISC-V cuenta con instrucciones que facilitan la realización de otros cálculos aritméticos. Todos los operandos han de ser registros y el primero de ellos siempre actúa como destino del resultado:

- `sub destino, minuendo, sustraendo`: resta sustraendo de minuendo y deja el resultado en destino.
- `mul destino, multiplicando, multiplicador`: multiplica multiplicando por el multiplicador.
- `div destino, dividendo, divisor`: efectúa la división entera de dividendo entre divisor y devuelve el cociente.
- `rem destino, dividendo, divisor`: efectúa la división entera de dividendo entre divisor y devuelve el resto.

Observamos que hay dos instrucciones de suma (`add` y `addi`) que difieren en el tipo del tercer argumento que toman. En realidad ambas realizan la misma operación, la suma, con dos modos de direccionamiento distintos, aspecto que tratamos en la Sección 1.5. El resto de instrucciones aritméticas no cuentan con la segunda variante que toma como parámetro un valor constante.

1.4 Carga y almacenamiento de datos

Hasta ahora, todos los valores con los que hemos operado eran inmediatos o estaban ya en registros. En la mayoría de casos, sin embargo, los datos con los que opera un programa están almacenados en memoria, separados del código. Es decir, no aparecen como valores constantes (o literales) en las propias instrucciones del programa. Asimismo, lo habitual es que los resultados obtenidos se alojen también en memoria. El procedimiento habitual para acceder a una posición de memoria, ya sea para leer o escribir, consiste en asignar a un registro dicha dirección y usarlo, a modo de puntero, para cargar (leer) o almacenar (escribir) un valor.

1.4.1 Espacio de direccionamiento y mapa de memoria

La ISA RV32IM contempla un tamaño de palabra de 32 bits, así como un espacio de direccionamiento de 32 bits. Esto implica que hay disponibles 2^{32} posiciones de memoria, cada una de las cuales tiene 1 byte (8 bits) de capacidad: la memoria del simulador tiene, por tanto, una capacidad total de **4 GB** (2^{32} bytes = 4096 MB). En ese espacio hay que almacenar el código del programa a ejecutar, los datos sobre los que operará, reservar una zona para dispositivos de entrada/salida, etc.

Las direcciones de memoria suelen notarse en base hexadecimal. Dado que cada dígito hexadecimal representa 4 bits, se precisan 8 posiciones en total para una dirección de 32 bits, de forma que la primera dirección (la más baja) es `0x0000 0000` y la última es `0xFFFF FFFF`.

En el simulador Ripes el mapa de memoria es el representado en la Figura 1.1. En el lateral izquierdo se ha dividido el espacio total de 4 GB en 16 porciones iguales de 256 MB. En el margen derecho se indican las direcciones de inicio, y de fin en su caso, de las secciones.

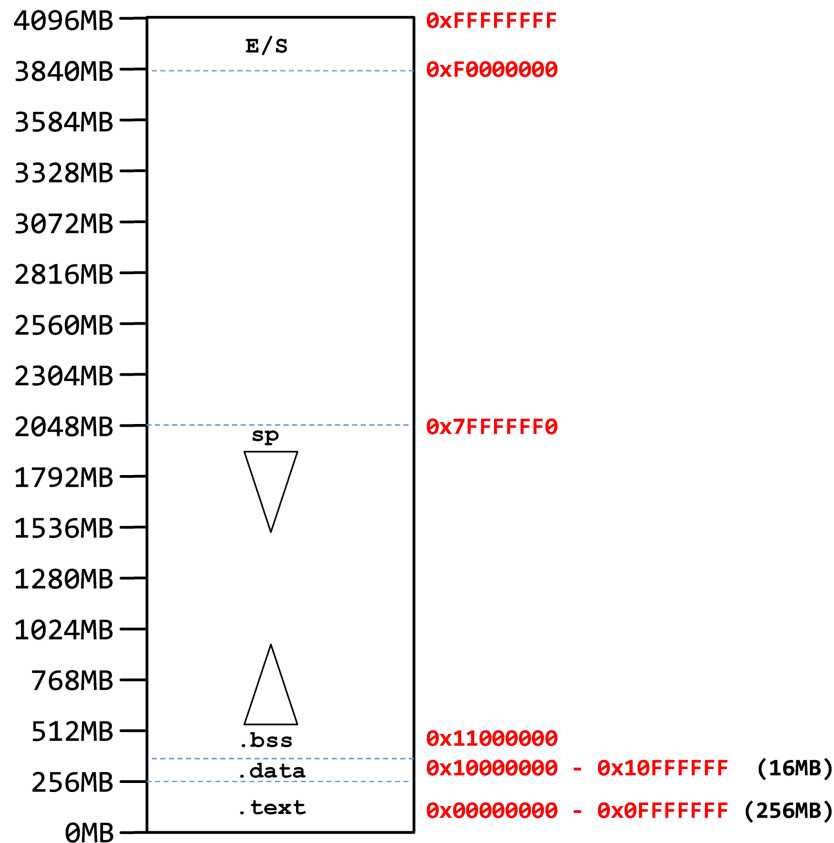


Figura 1.1: Mapa de memoria del emulador Ripes, con las distintas secciones en que se divide y sus tamaños. Figura tomada de [1].

La sección `.text` contendrá el código del programa. En las secciones `.data` y `.bss` se alojan los datos inicializados y sin inicializar, respectivamente. El registro `sp` apunta a la parte superior de la pila y, a medida que se alojan datos en esta, crece hacia abajo. Existe además una zona reservada para la entrada/salida mapeada, que trataremos en el Capítulo 3. La configuración del mapa de memoria de Ripes puede ajustarse con los parámetros del cuadro de diálogo al que da paso la opción *Edit*→*Settings*.

1.4.2 Leer y escribir datos en memoria

La lectura de un dato que está almacenado en memoria conlleva asignar la dirección de memoria a un registro y, a continuación, usarlo como puntero para leer el dato en sí mismo. Es una operación que se simplifica gracias a la seudoinstrucción `lw rd, símbolo`, en la que `rd` sería el registro de destino (donde se guardará el valor) y `símbolo` la etiqueta dispuesta delante de dicho dato al declararlo. Esta seudoinstrucción generará dos instrucciones reales, que puedes observar en el panel derecho de la vista *Editor* de Ripes.

La instrucción complementaria a `lw` es `sw`. Su finalidad es almacenar el contenido de un registro en la dirección de memoria que se especifique. A diferencia de la seudoinstrucción `lw rd, símbolo`, la forma de `sw` que usaremos es una instrucción real (`lw` también existe como instrucción real, como veremos en los aspectos avanzados de este capítulo). Para usarla emplearemos la sintaxis `sw ro, desp, rdir`, en la que `ro` es el registro cuyo valor quiere escribirse en memoria, `rdir` el registro que actuará como puntero y `desp` un desplazamiento a sumar a la dirección

contenida en `rdir`.

Antes de poder usar `sw`, por tanto, será preciso asignar al registro `rdir` la dirección de memoria que interese. Con este fin se usa la seudoinstrucción `la rdir, símbolo`. Como otras seudoinstrucciones, esta producirá dos instrucciones reales, que serán las que obtengan la dirección que corresponda a *símbolo* y la almacenen en el registro `rdir`.

Las instrucciones `lw` y `sw` operan sobre números enteros de 32 bits. Existen las variantes `lb` y `lh` para cargar números de 8 y 16 bits (`lb` y `lh` los cargan sin signo), con sus correspondientes `sb` y `sh` para almacenamiento. Las variantes de 64 bits, `ld` y `sd`, solo existen en RV64.

Sobre la base de las instrucciones `lw`, `sw` y `la`, el Listado 1.2 carga dos valores de una zona de memoria inicializada, los suma y almacena el resultado en una zona de memoria sin inicializar.

```
1  # a1: primer valor leído de memoria (n1)
2  # a2: segundo valor leído (n2)
3  # a0: suma de ambos
4  # t0: puntero a la dirección de res
5  # a7: código de servicio para ecall
6      .data      # Inicio del segmento de datos
7  n1: .word 7      # Almacenamos en memoria
8  n2: .word 3      # dos valores enteros
9
10     .bss       # Inicio de datos sin inicializar
11  res: .word 0
12
13     .text      # Inicio del segmento de código
14  lw a1, n1      # a1 = (n1)
15  lw a2, n2      # a2 = (n2)
16  add a0, a1, a2  # a0 = a1 + a2
17  la t0, res      # t0 -> res
18  sw a0, 0, t0    # (res) = a0
19  addi a7, zero, 10
20  ecall          # Llamar a la función 10 (exit)
```

Listado 1.2: Leer dos números de memoria y escribir su suma.

Las directivas `.data`, `.bss` y `.text` establecen el punto en el que comienzan las secciones de datos inicializados, datos sin inicializar y código, respectivamente. Cada una de estas secciones o segmentos tiene una dirección de inicio predeterminada. La directiva `.word` reserva espacio para un entero de 32 bits; directivas análogas para otros tamaños son `.byte` (8 bits) y `.half` (16 bits), entre otras (`.long`, en cambio, es un sinónimo de `.word`).

1.4.3 Examinar la memoria en Ripes

Al ejecutar el anterior programa observaremos, en la lista de registros, que `a0` contiene el resultado de la suma y que `t0` ha cambiado su valor para apuntar a la dirección identificada por el símbolo `res`. Para observar el estado de la memoria, y verificar así los datos almacenados en ella, accederemos a la vista **Memory**. Su panel izquierdo muestra las direcciones de memoria y su contenido: el formato (hexadecimal, decimal, etc.) se elige con las opciones de la primera lista desplegable de la parte inferior, y la última facilita el acceso a la sección de memoria que interese. En el panel derecho se facilita el mapa de memoria con la longitud real de las secciones de código y datos.

Las Figuras 1.2 y 1.3 muestran esta vista tras ejecutar el programa del Listado 1.2. Con la lista desplegable *Go to section* saltamos directamente a la sección que interese y con *Display type*

elegimos el formato de visualización (*Signed* en las capturas, para ver los valores como enteros con signo). En la sección `.data` (Figura 1.2) aparecen los dos valores declarados con `.word`: el 7 de `n1` en la dirección `0x10000000` y el 3 de `n2` en `0x10000004`. En la sección `.bss` (Figura 1.3) se observa, en la dirección `0x11000000`, el valor 10 que la instrucción `sw` escribió en `res`: el resultado de la suma.

0x10000028	X	X	X	X	X
0x10000024	X	X	X	X	X
0x10000020	X	X	X	X	X
0x1000001c	X	X	X	X	X
0x10000018	X	X	X	X	X
0x10000014	X	X	X	X	X
0x10000010	X	X	X	X	X
0x1000000c	X	X	X	X	X
0x10000008	0	0	0	0	0
0x10000004	3	3	0	0	0
0x10000000	7	7	0	0	0
0x0ffffffc	X	X	X	X	X
0x0ffffff8	X	X	X	X	X
0x0ffffff4	X	X	X	X	X
0x0fffff0	X	X	X	X	X

Display type: Signed Go to register: Select Go to section: Select

Figura 1.2: Vista *Memory* sobre la sección `.data` tras ejecutar el programa del Listado 1.2: los valores 7 y 3 declarados en memoria, con *Display type* en *Signed* y el desplegable *Go to section* abierto.

Dedica un tiempo a experimentar con esta vista, cambiando entre las secciones de memoria para examinar el código máquina del programa y los datos inicializados y sin inicializar en distintos formatos.

1.4.4 Un nuevo tipo de dato: el texto

Hasta ahora solo hemos guardado números en memoria, pero los programas también manejan texto. La solución es sencilla: cada carácter se representa con un número, siguiendo el código **ASCII** (*American Standard Code for Information Interchange*). A la letra H le corresponde el `0x48`, a la o el `0x6F`, y así con cada símbolo. Como todos los códigos caben en un byte, una cadena de texto es simplemente una secuencia de bytes consecutivos en memoria.

Para declararlas contamos con la directiva `.string`, que almacena los códigos ASCII de la cadena byte a byte y añade al final un **byte a cero**, el *terminador nulo*, que marca dónde acaba el texto. Así, la declaración `hola: .string "Hola\n"` deja en memoria estos seis bytes (`\n` es el carácter de salto de línea, código `0x0A`):

Carácter	H	o	l	a	\n	fin
Byte	0x48	0x6F	0x6C	0x61	0x0A	0x00

Para mostrar una cadena por la consola se usa el servicio 4 de `ecall`, siguiendo la convención de registros que vimos en la Sección 1.2.3: se carga en `a0` la *dirección* donde empieza la cadena

0x1100001c	X	X	X	X	X
0x11000018	X	X	X	X	X
0x11000014	0	0	0	0	0
0x11000010	X	X	X	X	X
0x1100000c	X	X	X	X	X
0x11000008	X	X	X	X	X
0x11000004	X	X	X	X	X
0x11000000	10	10	0	0	0
0x10fffffc	X	X	X	X	X
0x10fffff8	X	X	X	X	X
0x10fffff4	X	X	X	X	X
0x10fffff0	X	X	X	X	X
0x10ffffec	X	X	X	X	X
0x10ffffe8	X	X	X	X	X
0x10ffffe4	X	X	X	X	X

Display type: Signed Go to register: Select Go to section: Select

Figura 1.3: Vista *Memory* sobre la sección *.bss*: en la dirección `0x11000000` está el resultado de la suma (10) almacenado por `sw`.

(con `la`) y el servicio va imprimiendo caracteres hasta encontrar el terminador nulo. El Listado 1.3 muestra un programa completo.

```

1  # a0: direccion de la cadena a imprimir
2  # a7: codigo de servicio para ecall
3      .data
4
5  hola: .string "Hola, RISC-V\n"
6
7      .text
8
9  la a0, hola      # a0 = direccion donde empieza la cadena
10 li a7, 4         # servicio 4: imprimir cadena
11 ecall
12
13 li a7, 10        # servicio 10: terminar
14 ecall

```

Listado 1.3: Declarar una cadena de texto y mostrarla por la consola.

Tras ejecutarlo, abre la vista *Memory* y examina la sección *.data* byte a byte (columnas *Byte 0–3*): verás los códigos ASCII de cada carácter, empaquetados de cuatro en cuatro dentro de cada palabra, y el cero final. Y como cada carácter es un byte, la instrucción `lb` permite cargar uno suelto en un registro.

1.5 Modos de direccionamiento

Al presentar `add` y `addi` adelantábamos que, en realidad, son una misma operación con dos modos de direccionamiento distintos. Veamos qué significa esto. La mayor parte de instrucciones ensamblador precisan uno o más operandos (datos de entrada sobre los que se efectuará una determinada operación) y el destino donde quiere almacenarse el resultado. Según el número de dichos operandos y su tipo, hablamos de tipos o modos de direccionamiento concretos. Los procesadores RISC suelen ofrecer menos modos de direccionamiento que los CISC, ya que muchas de las operaciones solo pueden efectuarse tras haber cargado los datos en registros.

1.5.1 Los cuatro modos de direccionamiento de RISC-V

Un modo de direccionamiento no es más que la respuesta a una pregunta: ¿de dónde obtiene el procesador cada operando? En RISC-V la respuesta puede ser una de estas cuatro [2]:

1. **Direccionamiento inmediato:** el operando es una **constante que viaja dentro de la propia instrucción**, en el campo `imm`. En `addi a0, a1, 5`, el 5 no hay que buscarlo en ningún sitio: forma parte de la instrucción.
2. **Direccionamiento por registro:** el operando está en un **registro**, y lo que lleva la instrucción es el *número* de ese registro. En `add a0, a1, a2`, los campos `rs1` y `rs2` indican qué dos registros del banco hay que leer.
3. **Direccionamiento indexado** (también llamado de *base + desplazamiento*): el operando está en **memoria**. Su dirección se calcula sumando el contenido de un registro (la dirección *base*) y una constante de la instrucción (el *desplazamiento*). En `lw a0, 8(t0)`, la dirección del dato es `t0 + 8`. Es el modo que emplean las cargas y los almacenamientos.
4. **Direccionamiento relativo al pc:** la constante de la instrucción no localiza un dato, sino que es un desplazamiento que se **suma al contador de programa** para obtener la dirección de destino de un salto. Es el modo de instrucciones como `beq`.

La Figura 1.4 representa gráficamente los cuatro modos: en cada uno, el campo sombreado de la instrucción indica de dónde procede el operando (la propia instrucción, el banco de registros, la memoria o el `pc`).

Los cuatro formatos de instrucción que se describen a continuación están ligados a estos modos de direccionamiento, aunque no uno a uno: un mismo formato puede servir a más de un modo.

1.5.2 Formato general de las instrucciones RV32I

La longitud de las instrucciones, en formato de código máquina que es el que puede procesar la CPU, depende del diseño de la ISA. En procesadores CISC es habitual que esa longitud sea variable, de manera que algunas instrucciones ocupan un solo byte, otras dos, tres... hasta 15 bytes en ocasiones. En contraposición, las arquitecturas RISC promueven el uso de una **longitud fija** para las instrucciones, ya que esto simplifica de manera considerable la implementación de la ISA.

En RV32IM todas las instrucciones se almacenan en memoria ocupando **32 bits**. Este es el código que aparece en la segunda columna del panel *Executable code* de la vista *Editor* de Ripes: siempre ocupa 8 dígitos hexadecimales y, dado que cada dígito corresponde a 4 bits, se tienen 32 bits. Esos 32 bits tienen que codificar no solo la operación que quiere llevarse a

1. Direccionamiento inmediato



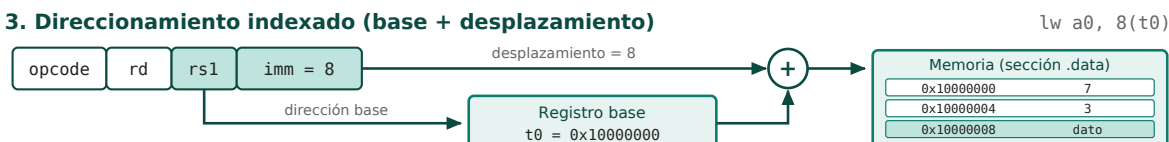
El operando (el 5) es una constante que viaja dentro de la propia instrucción: no hay que buscarlo en ningún otro sitio.

2. Direccionamiento por registro



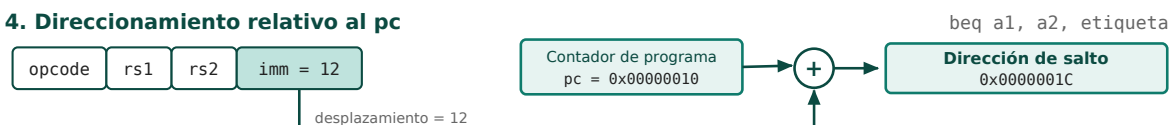
Los campos rs1 y rs2 indican qué registros del banco contienen los operandos (aquí, a1 y a2).

3. Direccionamiento indexado (base + desplazamiento)



La dirección del dato se calcula sumando a la dirección base (contenida en rs1 = t0) el desplazamiento imm: $0x10000000 + 8 = 0x10000008$; de esa posición de memoria (fila sombreada) se lee el dato. Es el modo de cargas y almacenamientos.

4. Direccionamiento relativo al pc



Si la condición se cumple, el pc pasa a valer pc + desplazamiento: $0x00000010 + 12 = 0x0000001C$, y la ejecución continúa por la instrucción almacenada en esa dirección de la sección .text.

Figura 1.4: Los cuatro modos de direccionamiento de RISC-V y el origen de los operandos en cada uno, con direcciones reales del mapa de memoria de Ripes: la sección .data comienza en 0x10000000 y la .text en 0x00000000. Elaboración propia a partir de [2].

cabo, sino también sus operandos (registro de destino, registros de origen, valor inmediato, etc.). Según el tipo de instrucción, los 32 bits se estructuran de una forma u otra. Cada categoría de instrucción, de las cuatro básicas que se describen a continuación, se asocia principalmente a un modo de direccionamiento. Para cada tipo incluiremos un ejemplo real de codificación, siguiendo a Patterson y Hennessy [2].

1.5.3 Instrucciones tipo R

Estas son instrucciones en las que todos los valores que actúan como operandos se encuentran alojados en registros, de ahí que se vinculen con el denominado **direccionamiento por registro**. El formato de la instrucción es el siguiente:

func7	rs2	rs1	func3	rd	opcode
31..25	24..20	19..15	14..12	11..7	6..0

Los tres registros implicados, rs1, rs2 y rd, aportan los operandos sobre los que se actuará y el destino en el que se almacenará el resultado, respectivamente. A cada uno le corresponden 5 bits ($2^5 = 32$) ya que hay un total de 32 registros. En todos los formatos de instrucción los registros siempre ocupan las mismas posiciones, lo cual facilita la decodificación por parte del microprocesador.

El código de operación, conocido como **opcode**, siempre ocupa los siete bits de menor peso de la instrucción. Hay ciertos grupos de instrucciones que tienen el mismo **opcode**, caso en el que la operación que realizan viene determinada por un código de función de 10 bits: los que ocupan las posiciones **func3** y **func7**. Un ejemplo de instrucción tipo R es `add a0, a1, a2`, así como cualquier otra operación aritmética o lógica cuyos operandos estén todos en registros. Las instrucciones `add`, `sub`, `and`, `or` y `xor`, entre otras, tienen asociado un mismo **opcode**, siendo la unión **func7+func3** la que determina la operación a ejecutar.

Como ejemplo real de codificación tomemos la instrucción `add x9, x20, x21`, que suma `x20` y `x21` y deja el resultado en `x9`. Usando los alias de la ABI (Tabla 1.1) se escribiría `add s1, s4, s5`. Su codificación es la siguiente:

0000000	10101	10100	000	01001	0110011
func7	rs2 = 21	rs1 = 20	func3	rd = 9	opcode

es decir, la palabra de 32 bits `0x015A04B3`.

El valor de cada campo es el siguiente:

- **opcode** = 0110011: identifica al grupo de instrucciones aritmético-lógicas de tipo R.
- **rd** = 01001 = 9: el registro de destino es `x9` (alias `s1`), donde se dejará el resultado.
- **func3** = 000 y **func7** = 0000000: unidos forman el código de función que, dentro del grupo que comparte este **opcode** (`add`, `sub`, `and`...), selecciona la operación `add`.
- **rs1** = 10100 = 20: el primer operando es el registro `x20` (alias `s4`).
- **rs2** = 10101 = 21: el segundo operando es el registro `x21` (alias `s5`).

1.5.4 Instrucciones tipo I y U

Este conjunto de instrucciones está asociado al **direccionamiento inmediato**, caracterizado porque uno de los operandos es un valor inmediato (un literal o constante que aparece en la propia instrucción) en lugar de un registro.

imm	rs1	func3	rd	opcode
31..20	19..15	14..12	11..7	6..0

Instrucción tipo I

imm	rd	opcode
31..12	11..7	6..0

Instrucción tipo U

En el tipo I el valor inmediato ocupa los cinco bits que en el tipo R corresponden al **rs2**, más los 7 bits de **func7**, por lo que en total se cuenta con **12 bits** para el valor inmediato. Ya conocemos una instrucción de este tipo: `addi a0, a1, 5`, en la que `a0` es el registro de destino (**rd**), `a1` el primer operando (**rs1**) y el valor literal 5 el segundo (**imm**). Hay varias más similares, todas ellas compartiendo un mismo **opcode**, como son `andi`, `ori` o `xori`.

Hay que tener presente que, en general, RISC-V procesa los valores inmediatos como números enteros **con signo**. El bit de mayor peso denota si el número es positivo o negativo, y los negativos se representan en complemento a 2 (Sección 0.2.3), de modo que con 12 bits el rango de valores posibles es $[-2048, 2047]$.

En RISC-V los números negativos se almacenan en complemento a 2. Por tanto, para obtener el negativo de cualquier número primero se invierten todos sus bits y, después, se suma 1. Considerando que tenemos 12 bits para representar el valor, el número 5 (0000 0000 0101)

pasaría a ser -5 tras invertir todos los bits (1111 1111 1010) y a continuación sumar 1 (1111 1111 1011).

También son de tipo I las instrucciones de carga, que emplean el **direccionamiento indexado**, consistente en tomar el contenido de un registro y sumarle el desplazamiento indicado por el valor inmediato, para obtener así el contenido de la dirección de memoria resultante. Es el caso de la instrucción `lw a0, 128, t0`, en la que al contenido del registro `t0` se le suma el desplazamiento 128 y, a continuación, se lee de `[t0 + 128]` una palabra para llevarla a `a0`.

Como ejemplo real de codificación de tipo I tomemos la instrucción `addi x9, x9, 1`, que suma 1 al registro `x9`. Con los alias de la ABI se escribiría `addi s1, s1, 1`. Su codificación es la siguiente:

0000000000001	01001	000	01001	0010011
imm = 1	rs1 = 9	func3	rd = 9	opcode

es decir, la palabra de 32 bits `0x00148493`.

El valor de cada campo es el siguiente:

- **opcode** = 0010011: identifica al grupo de instrucciones aritmético-lógicas con operando inmediato (`addi`, `andi`, `ori`...).
- **rd** = 01001 = 9: el registro de destino es `x9` (alias `s1`).
- **func3** = 000: dentro del grupo anterior selecciona la operación `addi`.
- **rs1** = 01001 = 9: el operando de origen es también `x9` (`s1`), al que se sumará el inmediato.
- **imm** = 0000000000001 = 1: el valor inmediato, un entero de 12 bits en complemento a 2.

El tipo U se caracteriza porque el valor inmediato tiene una longitud de **20 bits**, en lugar de 12, y porque el valor se interpreta como un entero **sin signo**. Además, solo se cuenta con un registro como operando (**rd**) que actúa como destino de la operación. Una instrucción de este tipo es `lui t0, valor`, en la que los 20 bits de *valor* se transfieren como los 20 bits de mayor peso del registro `t0`, mientras que los 12 bits de menor peso se ponen a 0. Esto equivale a desplazar 12 bits hacia la izquierda el valor o, lo que es lo mismo, multiplicarlo por 4096 ($2^{12} = 4096$).

La otra instrucción que emplea el formato U es `auipc rd, imm`. Esta permite usar un tipo de direccionamiento relativo en el que uno de los operandos es fijo: el contador de programa (registro `pc`). Es un tipo de direccionamiento que permite a los compiladores producir código que puede cargarse en cualquier posición de la memoria, ya que todas las referencias a datos y etiquetas son relativas al `pc`.

Como ejemplo real de codificación de tipo U tomemos la instrucción `lui x5, 0x12345`, que deja el registro `x5` con el valor `0x12345000`. Con los alias de la ABI se escribiría `lui t0, 0x12345`. Su codificación es la siguiente:

00010010001101000101	00101	0110111
imm = 0x12345	rd = 5	opcode

es decir, la palabra de 32 bits `0x123452B7`.

El valor de cada campo es el siguiente:

- `opcode` = 0110111: identifica a la instrucción `lui` (este formato no precisa códigos de función).
- `rd` = 00101 = 5: el registro de destino es `x5` (alias `t0`).
- `imm` = 0001 0010 0011 0100 0101 = 0x12345: los 20 bits del inmediato, que se transfieren a los bits 31..12 de `x5`; los 12 bits de menor peso quedan a 0, de ahí el resultado 0x12345000.

1.5.5 Instrucciones tipo S

Vinculado al conocido como **direccionamiento relativo**, este conjunto está formado por ciertas instrucciones de salto condicional. Se cuenta con dos operandos de entrada, alojados en sendos registros, sobre los cuales se llevará a cabo una operación, según cuyo resultado se modificará el `pc` sumándole el desplazamiento codificado en los 12 bits de `imm1` e `imm2` (en los saltos, esos bits representan el desplazamiento sin su bit 0, que siempre vale 0).

<code>imm1</code>	<code>rs2</code>	<code>rs1</code>	<code>func3</code>	<code>imm2</code>	<code>opcode</code>
31..25	24..20	19..15	14..12	11..7	6..0

Una instrucción que emplea este direccionamiento es `beq a0, a2, imm`, que compara el contenido de los registros `a0` y `a2` y, si es igual, modifica el `pc` en la magnitud indicada por el valor inmediato. En consecuencia, se produce un salto en el flujo de ejecución del programa. Conoceremos los saltos condicionales al final de este capítulo, en la Sección 1.7.

Este mismo reparto de campos es el que emplean las instrucciones de almacenamiento (`sb`, `sh`, `sw`), que también precisan dos registros de origen y no tienen registro de destino. Como ejemplo real de codificación tomemos la instrucción `sw x9, 240(x10)`, que almacena el contenido de `x9` en la dirección `x10 + 240`. Con los alias de la ABI se escribiría `sw s1, 240(a0)`. Su codificación es la siguiente:

0000111	01001	01010	010	10000	0100011
<code>imm1</code>	<code>rs2</code> = 9	<code>rs1</code> = 10	<code>func3</code>	<code>imm2</code>	<code>opcode</code>

es decir, la palabra de 32 bits 0x0E952823.

El valor de cada campo es el siguiente:

- `opcode` = 0100011: identifica al grupo de instrucciones de almacenamiento.
- `func3` = 010: dentro de ese grupo selecciona `sw` (almacenar una palabra de 32 bits; `sb` y `sh` usan otros valores).
- `rs1` = 01010 = 10: el registro `x10` (alias `a0`) actúa como puntero con la dirección base.
- `rs2` = 01001 = 9: el registro `x9` (alias `s1`) contiene el valor que se escribirá en memoria.
- `imm1` = 0000111 e `imm2` = 10000: concatenados forman 000011110000 = 240, el desplazamiento que se suma a la dirección base. No hay campo `rd`: sus bits los ocupa `imm2`.

En la codificación de los saltos condicionales, los bits del inmediato se reparten entre `imm1` e `imm2` en un orden ligeramente distinto al de los almacenamientos (Patterson y Hennessy denominan a esa variante formato SB [2]), pero la disposición de los campos es idéntica a la mostrada. Puedes comprobarlo en Ripes escribiendo una instrucción `beq` y observando su

código máquina en el panel derecho.

1.6 Aspectos avanzados

La seudoinstrucción `la`, que hemos usado para almacenar en un registro una dirección de memoria, produce dos instrucciones reales. Lo mismo ocurre con la seudoinstrucción `lw` cuando se usa solo con dos argumentos (el registro de destino y la dirección del dato a cargar), no así con la instrucción real del mismo nombre que, como ya sabemos, toma un parámetro adicional y puede escribirse como `lw rd, desp, rbase` o bien como `lw rd, desp(rbase)`. Esto no es óptimo en un programa que realice muchos accesos a memoria. Veamos la vía para optimizar el código en estos casos.

1.6.1 Asignación de una dirección a un registro

¿Por qué hacen falta dos instrucciones para poner una dirección en un registro? El truco está en los tamaños. Una instrucción ocupa exactamente 32 bits y, dentro de ella, además de la dirección tienen que caber el `opcode` y los registros: es imposible alojar ahí una dirección completa de 32 bits. El inmediato más grande disponible es el de `lui`, con 20 bits, y los 12 restantes los aporta el inmediato de una segunda instrucción, `addi`.

La clave está en cómo carga `lui` su inmediato. No lo copia tal cual en el registro: lo **desplaza 12 posiciones a la izquierda**, de modo que esos 20 bits pasan a ocupar los bits 31..12 (equivale a multiplicar por $2^{12} = 4096$, como vimos en el formato de tipo U) y los 12 bits de menor peso quedan **a cero**. El registro contiene ya un valor de 32 bits con la parte alta de la dirección en su sitio y un hueco de ceros abajo. Y gracias a ese hueco, cuando el bit 11 de la dirección vale 0, como en el ejemplo que sigue, la suma de `addi` no produce ningún acarreo: cada bit del inmediato cae sobre un cero, así que sumar equivale, exactamente, a encajar la pieza baja en su lugar. Si ese bit vale 1, en cambio, `addi` interpreta sus 12 bits como un número negativo, porque su inmediato tiene signo (va de -2048 a 2047), y la parte alta debe llevar un 1 más para compensarlo; el ensamblador hace ese ajuste por su cuenta al traducir `la`.

Veámoslo con números. Si la etiqueta `dato` está en la dirección `0x10000123`, la seudoinstrucción `la t0, dato` equivale a:

```
lui t0, 0x10000      t0 = 0x10000 << 12 = 0x10000000 (12 ceros abajo)
addi t0, t0, 0x123    t0 = 0x10000000 + 0x123 = 0x10000123
```

En binario se ve que la suma es un simple encaje de las dos piezas:

0001 0000 0000 0000 0000 0000 0000 0000	(<code>lui</code> : los 20 bits altos y el hueco de ceros)
+ 0000 0000 0000 0000 0000 0001 0010 0011	(<code>addi</code> : los 12 bits bajos)
= 0001 0000 0000 0000 0000 0001 0010 0011	es decir, <code>0x10000123</code>

Puedes comprobarlo en el panel derecho del *Editor* de Ripes: al escribir `la` verás las dos instrucciones reales en que se expande (en Ripes, la pareja relativa al `pc`: `auipc` y `addi`).

Si vamos a cargar muchos datos desde memoria con la seudoinstrucción `lw`, una de las dos instrucciones que genera será reiterativa, ya que la primera siempre cargará una dirección base en un registro, mientras que la segunda aplicará un desplazamiento sobre la misma.

El Listado 1.4 muestra cómo usar las instrucciones `lui` y `addi` para cargar en el registro `t0` (que pretendemos usar como si fuese un puntero) la dirección de 32 bits que corresponde a la etiqueta `par`. Descomponemos la dirección de dicha etiqueta en dos partes, mediante los operadores `%hi` y `%lo`, la primera con los 20 bits de mayor peso y la segunda con los 12 de menor peso. Al sumar ambas partes, se compone la dirección que interesa. Esta nos sirve para leer los dos datos, y cualesquiera otros que pudieran existir, con la instrucción `lw`. Observa cómo para leer el primer dato se usa un desplazamiento 0 y en el segundo 4, dado que cada entero ocupa 4 bytes (32 bits).

```

1  # t0: puntero a la etiqueta par
2  # a1: primer valor leído, par[0]
3  # a2: segundo valor leído, par[1]
4  # a0: suma de ambos
5      .data
6
7  par: .word 7, 3 # Dos valores enteros en memoria
8
9      .text
10
11  lui t0, %hi(par)      # t0[31..12] = par[31..12]
12  addi t0, t0, %lo(par) # t0 = t0 + par[11..0]
13
14  lw a1, 0, t0          # a1 = 0(t0) = par[0]
15  lw a2, 4, t0          # a2 = 4(t0) = par[1]
16  add a0, a1, a2        # a0 = a1 + a2

```

Listado 1.4: Uso de una dirección base con distintos desplazamientos para cargar datos.

En Ripes, `%hi` y `%lo` toman literalmente los 20 bits altos y los 12 bajos de la dirección, sin el ajuste descrito antes: si el bit 11 de la dirección vale 1, `addi t0, t0, %lo(par)` da un error de ensamblado. Con `auipc` en lugar de `lui`, los operadores serían `%pcrel_hi` y `%pcrel_lo`, pero no basta con cambiar los nombres, porque ambas partes se calculan respecto a la dirección de la `auipc`: con la `addi` justo detrás, se escribiría `%pcrel_lo(par+4)`, que es como Ripes expande `la`. La seudoinstrucción `la` funciona siempre.

En el día a día no hará falta escribir `lui` y `addi` a mano: la seudoinstrucción `la` hace ese trabajo por nosotros, y combinada con el direccionamiento indexado resulta muy cómoda para recorrer datos consecutivos. El Listado 1.5 muestra las dos maneras habituales de acceder a un vector: con desplazamientos fijos sobre la dirección base (cada `.word` ocupa 4 bytes, así que el elemento `i` está en base `+4i`) o avanzando el propio puntero con `addi`, el patrón que usaremos en los bucles del Capítulo 2.

```

1  # t0: puntero al elemento actual del vector
2  # a0-a3: elementos leídos, vec[0] a vec[3]
3      .data
4
5  vec: .word 10, 20, 30, 40
6
7      .text
8
9  la t0, vec            # t0 apunta al primer elemento
10 lw a0, 0(t0)          # a0 = vec[0] = 10
11 lw a1, 4(t0)          # a1 = vec[1] = 20
12 lw a2, 8(t0)          # a2 = vec[2] = 30
13
14 # La alternativa: avanzar el puntero

```

```

15 addi t0, t0, 12    # t0 apunta ahora a vec[3]
16 lw a3, 0(t0)      # a3 = vec[3] = 40

```

Listado 1.5: Dos formas de recorrer un vector a partir de la dirección cargada con `la`: desplazamientos fijos o avance del puntero.

1.7 Tomar decisiones: los saltos condicionales

Todos los programas que hemos escrito hasta aquí son secuenciales: sus instrucciones se ejecutan en orden, de la primera a la última, y cada ejecución hace exactamente lo mismo. Muchos problemas, sin embargo, exigen ejecutar o dejar de ejecutar un cierto bloque de instrucciones según que se cumpla o no una determinada condición, de forma similar a las instrucciones `if (cond) cierto else falso` de los lenguajes de programación de alto nivel.

La pieza que lo hace posible es la **alteración del flujo de ejecución**. El procesador sabe qué instrucción toca ejecutar porque su dirección está en el contador de programa (el registro `pc`, cuyo valor se actualiza de forma automática a medida que se ejecutan las instrucciones). Las **instrucciones de salto** modifican ese registro y, con él, el punto del programa por el que continúa la ejecución. En el código fuente el destino del salto se simboliza con una **etiqueta**, como las que ya venimos usando para nombrar datos, lo que nos evita calcular direcciones a mano.

Los **saltos condicionales** llevan a cabo una comparación entre dos valores y, según su resultado, alteran o no el contador de programa. Las instrucciones disponibles son:

- `beq rs1, rs2, imm`: salta si `rs1 = rs2`.
- `bne rs1, rs2, imm`: salta si `rs1 ≠ rs2`.
- `bge rs1, rs2, imm`: salta si `rs1 ≥ rs2`.
- `bgeu rs1, rs2, imm`: como la anterior pero interpretando los valores sin signo.
- `blt rs1, rs2, imm`: salta si `rs1 < rs2`.
- `bltu rs1, rs2, imm`: como la anterior pero interpretando los valores sin signo.

El valor inmediato que todas estas instrucciones precisan como tercer argumento es un desplazamiento, positivo o negativo, que se sumará al contador de programa, de forma que si la condición se cumple se ejecuta la operación $pc \leftarrow pc + imm$. Todas estas instrucciones son del tipo S: los 12 bits de su inmediato codifican el desplazamiento en múltiplos de 2 bytes (el bit 0, que siempre vale 0, no se guarda), de modo que el destino del salto puede estar entre 4096 bytes antes y 4094 bytes después de la propia instrucción.

La igualdad o desigualdad de los dos valores, alojados en sendos registros, es indistinta de si estos se interpretan como números con o sin signo. Al comparar si uno es menor o mayor que el otro, sin embargo, sí hay que tomar en consideración la existencia de signo, de ahí que haya disponibles dos versiones.

1.7.1 La comparación, bit a bit

¿Cómo se comparan, a bajo nivel, los contenidos de dos registros? El procesador no ve números, sino dos secuencias de 32 bits, y la comparación se resuelve mirando esos bits. La igualdad es

lo más sencillo: dos registros son iguales si coinciden bit a bit en sus 32 posiciones, y por eso a `beq` y `bne` les da igual que los valores tengan signo o no. Para averiguar cuál es menor se procede como con los números decimales, de izquierda a derecha: decide el primer bit, empezando por el de mayor peso, en el que los dos valores se diferencian, y el menor es el que tiene ahí un 0. Así comparan `bltu` y `bgeu`, que interpretan los valores sin signo. Con signo, el bit de mayor peso es el bit de signo (Sección 0.2.3), y eso cambia la regla: si los dos valores tienen el mismo signo se comparan igual que sin signo, pero si lo tienen distinto, el menor es el que tiene un 1 en ese bit, porque es el negativo.

De ahí que el mismo par de registros pueda dar respuestas opuestas. Si `t0` contiene `0xFFFFFFFF` y `t1` contiene `0x00000001`, los dos valores se diferencian ya en el bit de mayor peso, que `t0` tiene a 1. Sin signo, `t0` es el mayor (4294967295 frente a 1), y `bltu t0, t1, etiqueta` no salta; con signo, ese mismo 1 indica que `t0` es negativo (vale -1), y `blt t0, t1, etiqueta` sí salta.

1.7.2 Un condicional en ensamblador

Gracias a estas instrucciones resulta posible implementar condicionales tal y como se aprecia en el Listado 1.6, en el que se ejecuta una instrucción u otra según que el valor en `a2` sea mayor o igual que el de `a1` o no. Hace falta un detalle más: para que el camino del sí no ejecute también el bloque del no, hay que saltarse este último siempre; la pseudoinstrucción `j etiqueta` hace justo eso, saltar de manera incondicional, sin comprobar condición alguna. La etiqueta `fin` marca el punto donde ambos caminos se reencuentran y continúa la ejecución del programa; en el ejemplo, imprimiendo el resultado y terminando.

```

1  # a1, a2: valores a comparar
2  # a0:      resultado, la diferencia positiva
3  # a7:      código de servicio para ecall
4      .text
5      li a1, 12                # Valores de ejemplo
6      li a2, 7
7
8      bge a2, a1, mayora2
9      sub a0, a1, a2           # a0 = a1 - a2
10     j fin
11 mayora2:
12     sub a0, a2, a1           # a0 = a2 - a1
13 fin:
14     li a7, 1                 # Imprimir a0
15     ecall
16     li a7, 10                # Terminar
17     ecall

```

Listado 1.6: Un condicional escrito en ensamblador RISC-V; equivale al código C `if (a1 > a2) a0 = a1 - a2; else a0 = a2 - a1;`

1.7.3 El salto por dentro: una suma al contador de programa

Con el Listado 1.6 delante podemos seguir el salto por dentro. Al ensamblarlo, Ripes coloca la primera instrucción en la dirección `0x00000000` y cada una de las siguientes, 4 bytes más allá de la anterior, porque todas ocupan 32 bits; cada `li` se convierte en una única `addi`, ya que todos sus valores (12, 7, 1 y 10) caben en el inmediato de 12 bits. La tabla siguiente reproduce, junto al código fuente, lo que muestra el panel derecho de la vista *Editor*:

Dirección	Código fuente	Instrucción real	Código máquina
0x00000000	li a1, 12	addi x11 x0 12	0x00C00593
0x00000004	li a2, 7	addi x12 x0 7	0x00700613
0x00000008	bge a2, a1, mayora2	bge x12 x11 12 <mayora2>	0x00B65663
0x0000000C	sub a0, a1, a2	sub x10 x11 x12	0x40C58533
0x00000010	j fin	jal x0 8 <fin>	0x0080006F
0x00000014	mayora2: sub a0, a2, a1	sub x10 x12 x11	0x40B60533
0x00000018	fin: li a7, 1	addi x17 x0 1	0x00100893
0x0000001C	ecall	ecall	0x00000073
0x00000020	li a7, 10	addi x17 x0 10	0x00A00893
0x00000024	ecall	ecall	0x00000073

En el código máquina no queda rastro de las etiquetas. El ensamblador ha sustituido `mayora2` por la **distancia en bytes** que hay desde la propia instrucción de salto hasta ella, $0x14 - 0x08 = 12$, y ese 12 es el inmediato de `bge`. Lo mismo ha hecho con `fin` en `j fin`, que en realidad es `jal x0 8`: $0x18 - 0x10 = 8$. El nombre entre ángulos, `<mayora2>`, lo añade Ripes para indicar adónde lleva el salto, pero no forma parte de la instrucción.

Cuando el `pc` llega a `0x00000008` toca ejecutar `bge x12 x11 12`, y lo primero es comparar, bit a bit, `x12 = 7` con `x11 = 12`. Los 24 bits de mayor peso valen 0 en los dos, así que basta mirar los 8 de menor peso, `0000 0111` y `0000 1100`: el primer bit en que se diferencian, empezando por la izquierda, es el de peso 8, que el 7 tiene a 0 y el 12 a 1. El 7 es, por tanto, el menor, y la condición $7 \geq 12$ no se cumple. Con esa respuesta, el procesador decide el nuevo valor del `pc`:

- Si la condición se cumpliera, al `pc` se le sumaría el desplazamiento: $0x00000008 + 12 = 0x00000014$, la dirección de `mayora2`.
- Como no se cumple, el `pc` avanza igual que con cualquier otra instrucción: $0x00000008 + 4 = 0x0000000C$, la instrucción siguiente.

Saltar, por tanto, no es más que sumar al `pc` el desplazamiento en lugar de los 4 bytes de siempre.

Con los valores del listado, la ejecución sigue en `0x0000000C`, donde `sub` deja en `a0` el resultado $12 - 7 = 5$. En `0x00000010` llega `jal x0 8`, el salto incondicional: aquí no hay nada que comparar, y el `pc` pasa siempre a $0x00000010 + 8 = 0x00000018$, la dirección de `fin`, de modo que el programa se salta el bloque de `mayora2` (el resto de lo que hace `jal` lo veremos en el capítulo siguiente).

Si se intercambian los valores iniciales (`li a1, 7` y `li a2, 12`), la pregunta pasa a ser si $12 \geq 7$. Esta vez la condición se cumple y el `pc` pasa a $0x00000008 + 12 = 0x00000014$: la ejecución salta directamente a `mayora2`, donde `a0` recibe también $12 - 7 = 5$, y continúa en `fin` sin necesidad de otro salto. Puedes comprobarlo en Ripes ejecutando el programa paso a paso mientras observas el `pc`; al terminar, *Instrs. retired*, en *Execution info*, marca 9 instrucciones con los valores del listado y 8 con los valores intercambiados.

El desplazamiento también puede ser negativo: si la etiqueta está antes que el salto, la suma hace retroceder al `pc`, algo que aprovecharemos para construir los bucles del capítulo siguiente.

Sobre la base de las anteriores instrucciones, RISC-V define múltiples pseudoinstrucciones que facilitan saltos con otras condiciones, como `bgt`, `bgtu`, `ble` y `bleu`, que toman los mismos argumentos que las instrucciones previas, y `beqz`, `bnez`, `bgez`, `blez` y `bgtz`, que comparan un único registro con cero (en la instrucción real que generan, uno de los dos registros es siempre `zero`). Introduce estas pseudoinstrucciones en el editor de Ripes y observa en el panel derecho el código real que generan.

Con esto ya sabemos tomar decisiones. La otra mitad del control del flujo, repetir un bloque de instrucciones mientras se cumpla una condición, se apoya en estos mismos saltos y la veremos al comienzo del capítulo siguiente.

Bucles y funciones en RISC-V

Al final del capítulo anterior aprendimos a tomar decisiones con los saltos condicionales. En esta segunda práctica completaremos el control del flujo de ejecución, que es lo que convierte una simple calculadora en un programa de verdad: con esos mismos saltos construiremos bucles; sobre ellos levantaremos las llamadas a subrutinas; completaremos el repertorio con las instrucciones lógicas, de desplazamiento y de comparación; y cerraremos con la pila, la estructura que permite que unas funciones llamen a otras sin perder el camino de vuelta ni los datos que deben conservar.

2.1 Introducción

Para llevar a cabo funciones complejas, un programa necesita dos herramientas de control del flujo de ejecución. La primera, ejecutar o dejar de ejecutar un bloque de instrucciones según se cumpla o no una condición (el `if` de los lenguajes de alto nivel), la resolvimos en la Sección 1.7 con los saltos condicionales. La segunda es repetir la ejecución de un bloque las veces que se precise, como hacen las instrucciones `for` o `while` de los lenguajes de alto nivel: los bucles, el punto de partida de este capítulo.

Ambas se apoyan en el mecanismo que ya conocemos, la alteración del flujo de ejecución a través del contador de programa, así que aquí recogeremos esa idea justo donde la dejamos.

2.2 Instrucciones de salto

Las instrucciones de salto las presentamos en la Sección 1.7: modifican el contenido del `pc` asignándole un valor que, por regla general, estará simbolizado por una **etiqueta** dispuesta en el código fuente. De las instrucciones enumeradas en la Tabla 1.2 forman este grupo ocho, a las que habría que sumar varias seudoinstrucciones con fines similares, divididas en dos subgrupos: las de salto condicional, que ya conocemos, y las de salto incondicional, que completaremos aquí. Además, antes de cambiar el `pc` es posible guardar en otro registro la dirección de la instrucción siguiente, por regla general con el objetivo de volver más tarde a ese mismo punto: esa idea, que veremos enseguida, es la que hace posibles las rutinas/funciones y el retorno desde las mismas.

2.2.1 De los condicionales a los bucles

Recordemos las piezas: `beq`, `bne`, `bge` y `blt` (con sus variantes sin signo `bgeu` y `bltu`) saltan a una etiqueta cuando se cumple la comparación, las seudoinstrucciones `bgt`, `ble`, `beqz` o `bnez` abrevian condiciones habituales, y `j` salta siempre. Con ellas implementamos el condicional del Listado 1.6.

De manera análoga podemos crear una estructura de repetición o **bucle**, para lo cual nos serviríamos de un salto condicional que volvería al inicio del cuerpo del bucle hasta que deje de cumplirse una cierta relación que, en el caso del Listado 2.1, es que el contenido de `a0` no sea cero.

```
1  # a0: contador del bucle y valor que se imprime
2  # a7: código de servicio para ecall
3  .text
4  li a0, 10
5  li a7, 1                # Servicio 1: imprimir entero
```



```

6 repite:
7     ecall                # Imprime a0
8     addi a0, a0, -1
9     bnez a0, repite
10
11    li a7, 10             # Terminar
12    ecall

```

Listado 2.1: Un bucle escrito en ensamblador RISC-V; equivale al código C `for(int a0=10; a0 > 0; a0--) printf("%d", a0);`

La seudoinstrucción `bnez` compara el contenido de `a0` con el registro `zero` y, si no coinciden, ejecuta el salto, de forma que el bucle se repetirá hasta que `a0` contenga el valor 0.

2.2.2 Saltos incondicionales

Como su propia denominación indica, estas instrucciones cambian el valor del contador de programa siempre que se ejecutan, sin que exista condición previa alguna. Hay dos instrucciones de salto incondicional disponibles:

- `jal rd, imm`: cuenta con dos operandos y su formato recuerda al tipo U, aunque es un tipo propio, el J (UJ en Patterson y Hennessy): su inmediato de 20 bits es un desplazamiento con signo, como el de los saltos condicionales, así que permite saltar hacia delante o hacia atrás. En `rd` se guardará la dirección de la instrucción siguiente (`pc + 4`), tras lo cual el `pc` se actualizará sumándole ese desplazamiento.
- `jalr rd, rs, imm`: es una instrucción de tipo I que cuenta con tres operandos. En `rd` se guardará la dirección de la instrucción siguiente (`pc + 4`), tras lo cual el `pc` se actualizará asignándole el resultado de la suma `rs + imm`. En la práctica esto permite saltar a cualquier punto de un programa, sin las limitaciones de los desplazamientos de 12 o 20 bits respecto a la posición actual del `pc`.

Como primer parámetro de estas instrucciones suele emplearse el registro `ra` (seudónimo de `x1`). La seudoinstrucción `jal imm` produce el código `jal x1, imm`, de forma que no es preciso indicar de manera explícita el registro de destino. También es posible saltar de manera incondicional a un punto del programa con la seudoinstrucción `j imm`, caso en el que se genera el código `jal x0, imm` y, por tanto, la dirección de retorno se descarta.

La diferencia fundamental entre `jal` y `jalr` consiste en que esta última no suma el desplazamiento al `pc`, sino que asigna a este el resultado de `rs + imm`. Dado que antes se puede asignar a `rs` cualquier valor (lo habitual es emplear instrucciones como `lui` y `auipc` con este fin), esta instrucción hace posible la transferencia de la ejecución a cualquier punto del espacio de direccionamiento, sin las limitaciones de los desplazamientos relativos, que solo alcanzan unos 4 KiB (saltos condicionales) o 1 MiB (`jal`) hacia delante o hacia atrás.

2.2.3 Llamadas a subrutinas

El hecho de que ambas instrucciones de salto incondicional, `jal` y `jalr`, guarden en un registro la dirección de la instrucción siguiente antes de saltar hace posible el retorno a ese punto. Como hemos visto, se toma el valor del `pc`, se le suma 4 y el resultado se guarda en el registro de destino. No hay más que usar la instrucción `jalr` y cambiar el registro de destino por el de origen, usando el valor 0 como desplazamiento para que la operación real sea $pc \leftarrow rs$.

Las seudoinstrucciones `call` y `ret` facilitan este tipo de operativa. La primera salta al punto indicado y deja en `ra` la dirección de retorno, la de la instrucción que sigue a la llamada (Ripes la traduce en dos instrucciones reales, `auipc` y `jalr`). La segunda salta a la dirección de retorno que hay en `ra`: es un `jalr` que guarda su propia dirección de retorno en `x0`, es decir, que la descarta.

El Listado 2.2 es un programa en el que se ha escrito una subrutina, a partir de la etiqueta `print`, a la que se llama dos veces de forma consecutiva, cada una con un valor distinto en `a1`: ahí está la gracia de las subrutinas, escribir el código una vez y reutilizarlo en cada llamada. Se usan las llamadas del sistema ofrecidas por Ripes para mostrar cadenas de caracteres (el servicio 4 que conocimos en el Capítulo 1) y números tanto en decimal como en hexadecimal.

```
1  # a1: numero a imprimir, el parametro de print
2  # a0: argumento de cada ecall (cadena o numero)
3  # a7: codigo de servicio para ecall
4  .data
5  dec: .string "Decimal: "
6  hex: .string "Hexadecimal: "
7  sep: .string " - "
8  endl: .string "\n"
9
10 .text
11 inicio:
12     li a1, 9                # Primera llamada: imprime el 9
13     call print
14
15     li a1, 255              # Segunda llamada: imprime el 255
16     call print
17
18     li a7, 10               # Terminar
19     ecall
20
21 print:
22     la a0, dec
23     li a7, 4
24     ecall
25     mv a0, a1
26     li a7, 1
27     ecall
28     la a0, sep
29     li a7, 4
30     ecall
31     la a0, hex
32     ecall
33     mv a0, a1
34     li a7, 34
35     ecall
36     la a0, endl
37     li a7, 4
38     ecall
39     ret
```

Listado 2.2: Programa con una rutina de impresión de números en decimal y hexadecimal.

Prueba el programa en Ripes y observa las dos líneas que produce (el 255 resulta especialmente ilustrativo en hexadecimal: `0x00ff`, porque Ripes rellena con ceros hasta tener al menos cuatro cifras; por eso el 9 aparece como `0x0009`). Añade alguna llamada más con otros valores; en cuanto combines esta idea con los bucles del Listado 2.1, podrás imprimir tablas enteras con muy poco

código.

Dado que la seudoinstrucción `call` siempre almacena la dirección de retorno en el registro `ra`, de donde la toma la seudoinstrucción `ret` para volver a asignarla al `pc`, no es posible usar estas seudoinstrucciones, por sí solas, para tener subrutinas anidadas (es decir, una función que llame a otra) porque el primer valor de `ra` se perdería y, con él, la posibilidad de devolver el control al punto adecuado.

2.3 Otras instrucciones de la arquitectura RV32IM

Con lo visto hasta aquí (aritméticas, cargas y almacenamientos, y ahora los saltos) ya podríamos escribir la gran mayoría de programas. Aun así, RV32IM ofrece unas cuantas instrucciones más que, en el momento oportuno, ahorran trabajo y ciclos. Las repasamos en este apartado antes de dar el salto a las funciones.

2.3.1 Instrucciones lógicas

Las operaciones lógicas entre bits son de uso muy habitual al programar a bajo nivel, ya que con ellas es posible activar un bit concreto, desactivarlo, invertirlo, comprobar si está activo o no, etc. Las instrucciones disponibles para ello son las siguientes:

- `and rd, rs1, rs2` y `andi rd, rs1, imm`: efectúa un AND bit a bit entre los operandos `rs1` y `rs2` (la primera) o bien `rs1` e `imm` (la segunda), dejando el resultado en `rd`.
- `or rd, rs1, rs2` y `ori rd, rs1, imm`: efectúa un OR bit a bit entre los dos operandos y guarda el resultado en `rd`.
- `xor rd, rs1, rs2` y `xori rd, rs1, imm`: efectúa un XOR bit a bit entre los dos operandos y guarda el resultado en `rd`.

¿Cómo comprobarías si el valor almacenado en un registro es par o impar? Bastaría con realizar una operación lógica AND con el valor adecuado. Todos los números impares tienen el bit de menor peso a 1, mientras que en los pares está a 0. Con `andi t0, a0, 1` tendríamos en `t0` el valor 0 si el número es par o 1 en caso contrario, por lo que con un salto condicional `beqz` podríamos discernir entre ambas posibilidades.

A estas hay que añadir un grupo de instrucciones que también operan al nivel de bit, pero en este caso con el objetivo de producir desplazamientos en un sentido u otro:

- `sll rd, rs1, rs2` y `slli rd, rs1, imm`: desplaza el valor contenido en `rs1` hacia la izquierda, tantos bits como indique `rs2` (la primera) o `imm` (la segunda), dejando el resultado en `rd`.
- `srl rd, rs1, rs2` y `srli rd, rs1, imm`: desplaza el valor contenido en `rs1` hacia la derecha, tantos bits como indique `rs2` o `imm`, y guarda el resultado en `rd`.
- `sra rd, rs1, rs2` y `srai rd, rs1, imm`: desplaza el valor contenido en `rs1` hacia la derecha, tantos bits como indique `rs2` o `imm`, tomando en consideración el bit de signo. El resultado se almacena en `rd`.

Las instrucciones `sll` y `srl` llevan a cabo un desplazamiento *lógico* de los bits, tratándolos a todos por igual, mientras que `sra` trata de forma especial el bit de signo, por lo que se dice que efectúa un desplazamiento *aritmético* en lugar de lógico.

El desplazamiento de un valor en un bit a la izquierda equivale a multiplicar dicho valor por dos. De forma análoga, desplazarlo a la derecha resulta en una división por dos: con `srl` si el número no tiene signo y con `sra` si lo tiene (con los negativos impares, `sra` redondea hacia abajo: -7 pasa a -4 , mientras que `div` da -3). Cambiando el número de bits es posible multiplicar/dividir por cualquier potencia de dos sin necesidad de recurrir a las instrucciones `mul` y `div`. Esto resulta interesante porque, en un procesador real, desplazar bits es mucho más rápido que multiplicar y, sobre todo, que dividir (en el procesador de ciclo único de Ripes, en cambio, todas las instrucciones tardan un ciclo).

2.3.2 Instrucciones de comparación

En la Sección 1.7 conocimos instrucciones que verifican si se cumple una cierta relación entre dos registros y, en consecuencia, alteran el flujo de ejecución. Además de esas, en RV32IM disponemos de cuatro instrucciones de comparación adicionales. La diferencia es que estas no alteran el valor del `pc`, sino que se limitan a almacenar el valor 1 o 0 en un registro de destino, según que el primer operando sea menor que el segundo o no, respectivamente.

Son cuatro instrucciones que hacen la misma comparación: `slt rd, rs1, rs2`, `sltu rd, rs1, rs2`, `slti rd, rs1, imm` y `sltiu rd, rs1, imm`. En todos los casos se comprueba si el contenido del registro `rs1` es menor que el almacenado en `rs2` o el valor inmediato `imm`. Las variantes `slt` y `slti` operan con números con signo, mientras que `sltu` y `sltiu`, las terminadas en `u` (de *unsigned*), lo hacen con números sin signo. Si intercambiamos el orden de los operandos a comparar, sería equivalente a tener una instrucción `sgt`, que comprobaría si el primero es mayor que el segundo.

Estas instrucciones son útiles cuando el resultado de la comparación no necesariamente implica cambiar el flujo de ejecución. Imaginemos que tenemos una lista de calificaciones y, recorriéndolas con un bucle, queremos contar cuántas de ellas son inferiores a 5. Podríamos hacerlo con instrucciones de salto, para comparar si el valor cargado desde memoria es inferior a 5 y, en consecuencia, desviar la ejecución a una instrucción que incremente un contador. La alternativa sería la mostrada en el Listado 2.3, en la que no existe dicho salto condicional.

```
1  # t0: puntero a la nota actual del vector
2  # t1: notas que quedan por procesar
3  # a2: nota leída de memoria
4  # a1: resultado de la comparación (1 si nota < 5)
5  # a0: conteo de notas menores que 5
6      .data
7
8  notas: .word 7, 5, 6, 3, 8, 2, 4, 6, 4, 9
9
10     .text
11
12     la t0, notas
13     li t1, 10
14     li a0, 0
15
16  bucle:
17     lw a2, 0, t0
18     slti a1, a2, 5
```

```

19      add a0, a0, a1
20
21      addi t1, t1, -1
22      addi t0, t0, 4
23      bnez t1, bucle
24
25      li a7, 1           # Imprimir el conteo (a0)
26      ecall
27      li a7, 10          # Terminar
28      ecall

```

Listado 2.3: Contar el número de valores que son inferiores a uno dado.

2.3.3 Otras pseudoinstrucciones

En esta y la práctica anterior hemos conocido la mayoría de pseudoinstrucciones contempladas en la ISA RV32IM. En realidad, solo hay tres más de uso relativamente común:

- **not rd, rs**: calcula el complemento a uno de **rs** (invirtiendo el estado de todos sus bits) y lo almacena en **rd**.
- **neg rd, rs**: calcula el complemento a dos de **rs** (el complemento a uno más 1) y lo almacena en **rd**.
- **nop**: no realiza ningún trabajo útil, simplemente consume uno o más ciclos de reloj.

2.4 Aspectos avanzados: la pila y las funciones

Ya conocemos las instrucciones de salto, mediante las cuales es posible realizar llamadas a funciones desde cualquier punto de un programa y, cuando terminan su ejecución, volver a la instrucción correspondiente. Este es un aspecto fundamental para los lenguajes de alto nivel. Además de la instrucción de salto incondicional, encargada de llevar el punto de ejecución al inicio de la función a la que se invoca, los lenguajes de alto nivel siguen unas **convenciones de llamada** que determinan, entre otros aspectos, los registros que han de utilizarse para facilitar parámetros a una función, al llamarla, o los empleados para devolver los valores de retorno. Es un escenario en el que se utiliza una estructura de datos básica: la **pila**.

2.4.1 La pila como estructura de datos

Antes de ver cómo la usa el procesador, conviene entender qué es una pila por sí misma. Una **estructura de datos** es una forma de organizar un conjunto de datos junto con las operaciones que se permiten sobre ellos. La **pila** (*stack*) es una de las más sencillas y se comporta exactamente como una pila de platos: cada plato nuevo se coloca encima de los demás y, cuando hace falta uno, se coge el de arriba. Nadie saca un plato de en medio.

Esa idea se traduce en dos operaciones:

- **Apilar** (*push*): colocar un dato encima de todos los demás.
- **Desapilar** (*pop*): retirar el dato que está encima de todos y recuperar su valor.

La posición del último dato apilado se llama **tope** (o cima) de la pila, y es la única accesible: para llegar a un dato que está más abajo hay que desapilar antes todos los que tiene encima. En consecuencia, los datos salen en el orden inverso al que entraron: el último en entrar es el primero en salir, comportamiento que se conoce como **LIFO** (*Last In, First Out*). La Figura 2.1 lo muestra con tres números: tras apilar 5, 8 y 3, el primero en salir es el 3 y el siguiente, el 8.

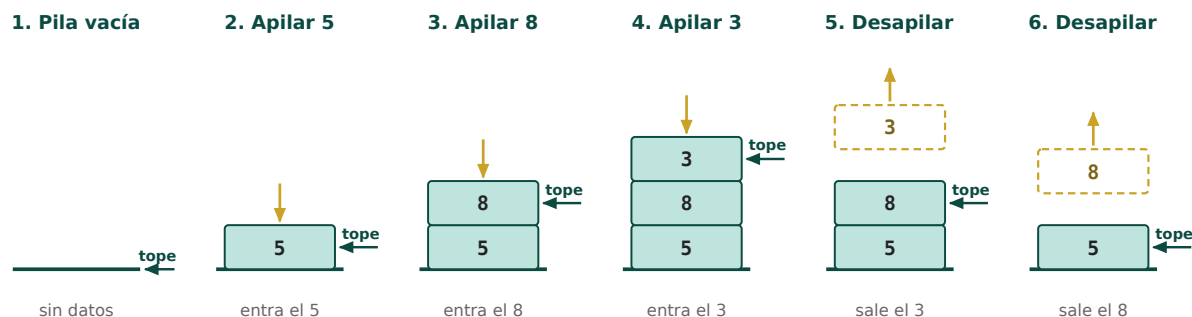


Figura 2.1: La pila como estructura de datos: cada operación de apilar deja el dato nuevo en el tope y cada operación de desapilar retira el que está en el tope, de modo que los datos salen en orden inverso al de entrada. Elaboración propia.

Una cola, como la de la caja de un supermercado, funciona justo al revés: sale primero quien llegó primero. La pila, en cambio, es la estructura adecuada siempre que haya que deshacer un camino en orden inverso, como hace la opción de deshacer de un editor de texto, que anula primero el último cambio. Y eso es precisamente lo que ocurre con las funciones: si el programa principal llama a una función A y esta, a su vez, llama a otra función B, B termina antes que A, y A antes que el programa principal. Lo que cada función guarda al empezar se recupera, por tanto, en orden inverso, que es justo el orden en que salen los datos de una pila.

En el procesador, la pila no es un circuito aparte, sino una zona de la memoria principal, y el repertorio RV32IM no tiene instrucciones específicas para apilar y desapilar: se hace con `addi`, `sw` y `lw` sobre el registro `sp` (*stack pointer*, puntero de pila), que marca en todo momento el tope. Para apilar se reserva espacio y se guarda el dato; para desapilar se recupera el dato y se libera el espacio, como veremos a continuación. Antes, un aviso que evita muchas confusiones: en memoria, la pila de RISC-V crece hacia las direcciones más bajas, de modo que en la Figura 2.2 aparece «boca abajo», con el tope en la parte inferior. Funciona, sin embargo, exactamente igual que la de los platos.

2.4.2 Almacenamiento temporal de datos en la pila

Cuando se llama a una función es con el objetivo de que realice un cierto trabajo. Para ello, se verá obligada a usar algunos registros. No obstante, el código de la función debe tener presente que el programa desde el que se la ha llamado espera que el contenido de determinados registros no cambie, por lo que no debería alterarlos o, en su defecto, debe preservarlos a fin de restaurarlos antes de devolver el control.

Una función puede modificar cualquiera de los registros temporales, `t0` a `t6`, y también los registros `a0` a `a7` que, como se ha explicado, sirven como parámetros de llamada y retorno, además de `ra`, que cada `call` sobrescribe. Si el código que invoca a una función tiene interés en el contenido de alguno de esos registros, debe encargarse por sí mismo de preservarlos. Los registros `s0` a `s11`, en cambio, han de ser conservados, igual que el valor de `sp`, lo que implica que la función debe, en caso de que necesite utilizarlos, guardar su contenido y restaurarlo al final de la misma.

Colocando el puntero del ratón sobre cualquier registro, en el panel GPR de Ripes, aparecerá una descripción de su finalidad y, en todos salvo **zero**, **gp** y **tp**, una segunda línea con la indicación *Saver: Caller* o *Saver: Callee*. En el primer caso el registro ha de ser preservado por el código que llama a la función, mientras que en el segundo es esta la que asume esa responsabilidad.

Dado que el almacenamiento del contenido de esos registros es una necesidad temporal (solo se precisa mientras dura la ejecución de la función a la que se llama), se usará para ello una estructura de memoria diseñada a tal efecto: la pila. El registro **sp** es la piedra angular para su uso: contiene siempre la dirección del tope de la pila (la de la última posición reservada) en una estructura que crece en memoria hacia abajo, a medida que en ella se guardan datos. La Figura 2.2 muestra ese crecimiento con un ejemplo mínimo: guardar dos registros. Fíjate en que, aunque entre los dos solo ocupan 8 bytes, se reservan 16, es decir, cuatro palabras, de las que dos quedan sin usar. No es un descuido: la convención de llamada de RISC-V exige que la dirección en **sp** sea siempre un **múltiplo de 16**, de modo que cualquier reserva se redondea hacia arriba hasta el siguiente múltiplo de 16 bytes (8 bytes pasan a ser 16, y 20 bytes, 32). Así, cualquier función, la haya escrito quien la haya escrito, encuentra la pila alineada de la misma manera.



Figura 2.2: El crecimiento de la pila en tres pasos: al reservar espacio, **sp** baja (resta) y, con los desplazamientos sobre **sp**, los registros quedan guardados en el hueco reservado. Elaboración propia.

En general, cuando una función necesite preservar registros, deberemos ajustar su código al siguiente esquema, por orden:

1. **Reservar** en la pila el espacio necesario para almacenar en ella los registros a preservar. Dado que la pila crece hacia abajo, de direcciones superiores a inferiores, esto implica restar al registro **sp** un cierto valor. Este estará en consonancia con la cantidad de información a guardar, pero debe tenerse en cuenta que la dirección en **sp** siempre ha de ser un **múltiplo de 16**.
2. **Guardar** el contenido de los registros apropiados en el espacio reservado en el paso previo. Para ello se usará el registro **sp** y un desplazamiento.
3. **Realizar la tarea** que la función tenga encomendada. Si a lo largo de esta parte del código se necesitase el valor original de un registro alterado, siempre puede recuperarse desde la pila.
4. **Restaurar** el contenido de los registros que se guardaron en el paso 2, cargándolos con los mismos desplazamientos sobre el registro **sp**.

5. **Ajustar** el registro `sp` para liberar el espacio que se reservó en el paso 1, lo cual deja la pila en el estado correcto para devolver el control.

Veámoslo con un caso en el que la pila hace falta de verdad: una función que llama a otra. Supongamos que queremos calcular $a^2 + b^2$ con una función, `suma_cuadrados`, que recibe a y b en `a0` y `a1` y devuelve el resultado en `a0`, y que para elevar al cuadrado se apoya en otra función más pequeña, `cuadrado`. El Listado 2.4 muestra las dos, junto con un programa principal que las usa; los comentarios numerados señalan los pasos del esquema anterior.

```
1  # a0, a1: los dos numeros; a0: resultado de suma_cuadrados
2  # s0: primer cuadrado, que debe sobrevivir a la segunda llamada
3  # s1: segundo numero, que debe sobrevivir a la primera llamada
4  # ra: direccion de retorno, que cada call sobrescribe
5  main:
6      li a0, 3
7      li a1, 4
8      call suma_cuadrados    # a0 = 3*3 + 4*4
9      li a7, 1               # Imprimir el resultado
10     ecall
11     li a7, 10              # Terminar
12     ecall
13
14 # Funcion suma_cuadrados(a, b) -> a0 = a*a + b*b
15 suma_cuadrados:
16     addi sp, sp, -16        # 1. Reservar sitio en la pila
17     sw ra, 0(sp)           # 2. Guardar ra, s0 y s1
18     sw s0, 4(sp)
19     sw s1, 8(sp)
20
21     mv s1, a1              # 3. El trabajo: b, a salvo en s1
22     call cuadrado          #    a0 = a*a
23     mv s0, a0              #    primer cuadrado, a salvo en s0
24     mv a0, s1
25     call cuadrado          #    a0 = b*b
26     add a0, a0, s0         #    a0 = a*a + b*b
27
28     lw s1, 8(sp)           # 4. Restaurar los registros
29     lw s0, 4(sp)
30     lw ra, 0(sp)
31     addi sp, sp, 16        # 5. Liberar el espacio
32     ret
33
34 # Funcion cuadrado(x) -> a0 = x*x
35 cuadrado:
36     mul a0, a0, a0
37     ret
```

Listado 2.4: Función `suma_cuadrados()`, que llama dos veces a la función `cuadrado()`, y programa que la usa, a partir de la etiqueta `main`.

La función `cuadrado` no necesita la pila: no llama a ninguna otra y solo modifica `a0`, donde deja su resultado. En `suma_cuadrados`, en cambio, cada registro que se guarda tiene un motivo concreto:

- `ra`, porque cada `call` escribe en él la dirección a la que debe volver la función llamada. Sin guardarlo antes, `suma_cuadrados` perdería la suya y no sabría volver a `main`. Es el

problema de las subrutinas anidadas que anunciaba la nota que sigue al Listado 2.2, y la pila lo resuelve.

- **s1**, porque *b* llega en **a1**, un registro que **cuadrado** tiene derecho a modificar. Para que sobreviva a la primera llamada se copia en **s1**, que **cuadrado** está obligada a respetar.
- **s0**, porque el primer cuadrado tiene que sobrevivir, del mismo modo, a la segunda llamada.

Como quien llama a **suma_cuadrados** espera encontrar **s0** y **s1** intactos, la función guarda al empezar sus valores anteriores y los restaura antes de volver. Son tres registros, 12 bytes, pero se reservan 16 para que **sp** siga siendo múltiplo de 16. Con 3 y 4, la consola muestra 25.

2.4.3 Una función que llama a otra dentro de un bucle

El mismo esquema sirve cuando la llamada se repite en cada vuelta de un bucle. El Listado 2.5 define la función **imprime_lista**, que recibe en **a0** la dirección de una lista de números y en **a1** cuántos tiene, y los imprime uno por línea llamando en cada vuelta a otra función, **imprime**.

```
1  # a0: direccion de la lista y, luego, numero que se imprime
2  # a1: cuantos numeros tiene la lista
3  # s0: puntero al numero actual, que debe sobrevivir a cada llamada
4  # s1: numeros que quedan, que debe sobrevivir a cada llamada
5  # a7: codigo de servicio para ecall
6  .data
7  lista: .word 12, 7, 30
8  n:     .word 3
9  endl:  .string "\n"
10
11  .text
12  main:
13      la a0, lista
14      lw a1, n
15      call imprime_lista
16      li a7, 10          # Terminar
17      ecall
18
19  # Funcion imprime_lista(direccion, n): un numero por linea
20  imprime_lista:
21      addi sp, sp, -16    # 1. Reservar sitio en la pila
22      sw ra, 0(sp)       # 2. Guardar ra, s0 y s1
23      sw s0, 4(sp)
24      sw s1, 8(sp)
25
26      mv s0, a0          # 3. El trabajo: puntero en s0
27      mv s1, a1          # y contador en s1
28  bucle:
29      lw a0, 0(s0)       # Numero actual
30      call imprime       # imprime puede cambiar a0-a7 y t0-t6
31      addi s0, s0, 4
32      addi s1, s1, -1
33      bnez s1, bucle
34
35      lw s1, 8(sp)       # 4. Restaurar los registros
36      lw s0, 4(sp)
37      lw ra, 0(sp)
38      addi sp, sp, 16    # 5. Liberar el espacio
```

```

39     ret
40
41 # Funcion imprime(x): escribe x y un salto de linea
42 imprime:
43     li a7, 1
44     ecall
45     la a0, endl
46     li a7, 4
47     ecall
48     ret

```

Listado 2.5: Función `imprime_lista()`, que llama a la función `imprime()` en cada vuelta de un bucle, y programa que la usa, a partir de la etiqueta `main`.

El puntero al número actual y el contador de los que quedan tienen que conservar su valor de una vuelta a la siguiente, es decir, sobrevivir a cada llamada a `imprime`. En un registro temporal o de argumentos, como `t0` o `a1`, no estarían a salvo, porque `imprime` puede modificarlos (de hecho, cambia `a0` y `a7`); por eso van en `s0` y `s1`, e `imprime_lista` preserva estos dos registros en la pila junto a `ra`, que el `call` de cada vuelta sobrescribe. La consola muestra los tres números, 12, 7 y 30, cada uno en su línea.

La pila tiene todavía un uso más en las funciones, aunque es poco habitual: si una función necesitara más de ocho parámetros, los que no caben en los registros de `a0` a `a7` se le pasarían a través de la pila.

2.4.4 Guardar datos de forma provisional: invertir una lista

Hasta aquí hemos usado la pila dentro de las funciones, pero su comportamiento LIFO es útil por sí mismo, en cualquier punto de un programa, siempre que haga falta guardar unos datos de forma provisional para recuperarlos después en orden inverso. El Listado 2.6 lo aprovecha para mostrar una lista de números al revés, y hace con instrucciones lo que muestra la Figura 2.1, pero hasta vaciar la pila: apila el 5, el 8 y el 3, y después los desapila todos.

```

1 # t0: puntero al numero actual de la lista
2 # t1: numeros que quedan por apilar o por desapilar
3 # t2: numero leído de la lista
4 # a0: numero desapilado, que se imprime
5 # a7: codigo de servicio para ecall
6     .data
7     lista: .word 5, 8, 3           # Los numeros, en el orden en que se apilan
8     n:     .word 3                # Cuantos hay
9     esp:   .string " "
10
11     .text
12     la t0, lista
13     lw t1, n
14     apila:
15         lw t2, 0(t0)              # Leer el numero de la lista
16         addi sp, sp, -16          # Reservar sitio en la pila
17         sw t2, 0(sp)              # y guardarlo en el tope (apilar)
18         addi t0, t0, 4            # Pasar al siguiente numero
19         addi t1, t1, -1
20         bnez t1, apila
21
22     lw t1, n
23     desapila:

```

```

24     lw a0, 0(sp)           # Recuperar el numero del tope
25     addi sp, sp, 16        # y liberar su sitio (desapilar)
26     li a7, 1              # Imprimirlo
27     ecall
28     la a0, esp             # seguido de un espacio
29     li a7, 4
30     ecall
31     addi t1, t1, -1
32     bnez t1, desapila
33
34     li a7, 10              # Terminar
35     ecall

```

Listado 2.6: Mostrar una lista de números en orden inverso con ayuda de la pila.

El programa tiene dos bucles. El primero recorre la lista y apila cada número: reserva sitio restando 16 a `sp` y guarda el dato en el tope con `sw`. Cada número ocupa solo 4 bytes, pero se apila de 16 en 16 para que `sp` siga siendo múltiplo de 16, la misma regla que se sigue en las funciones. El segundo bucle hace el camino contrario: lee el dato del tope con `lw`, libera su sitio sumando 16 a `sp` y lo imprime. Como el último en entrar es el primero en salir, en la consola aparece `3 8 5`. Fíjate, además, en que el segundo bucle deshace exactamente lo que hizo el primero: al terminar, `sp` vuelve a tener el valor del principio, algo que siempre debe cumplirse cuando se usa la pila.

La pila tiene también un uso interno que conviene conocer, aunque aquí no lo programaremos: es la pieza que hace posibles las **funciones recursivas**, las que se llaman a sí mismas. Cada llamada apila su dirección de retorno y sus propios datos antes de hacer la siguiente y, al terminar, los recupera del tope. Así, por muchas llamadas que se encadenen, cada una sabe a qué punto debe volver y con qué valores, y el programa no se pierde.

Operaciones de entrada y salida

Con lo aprendido hasta aquí, un programa puede calcular y tomar decisiones, pero sigue encerrado en sí mismo: no percibe nada del exterior ni muestra su trabajo más allá de lo que escribimos en la consola. En esta tercera práctica le daremos ese sentido que le falta y aprenderemos a comunicar el procesador con dispositivos de **entrada y salida**. Veremos primero los tres que ofrece Ripes (una matriz de leds, un panel de microinterruptores y un conjunto de pulsadores); comprobaremos después que hablar con ellos no exige instrucciones nuevas, sino las mismas cargas y almacenamientos del Capítulo 1, gracias a la entrada/salida mapeada en memoria; y cerraremos viendo cómo esa misma idea se traslada al hardware real, desde un microcontrolador hasta un ordenador con sistema operativo.

3.1 Introducción

Una CPU por sí sola no compone un computador. Este precisa memoria, algo que ya sabemos cómo usar con las instrucciones para cargas y almacenamientos, y también comunicarse con dispositivos de **entrada y salida**, mediante los cuales el computador interacciona con el mundo exterior y, en particular, con los usuarios.

Los dispositivos de E/S serán de diferente índole, según el tipo de computador de que se trate. En un ordenador personal los habituales son el teclado, la pantalla y el ratón. En un servidor en la nube no se cuenta con esos elementos, pero la conexión de red (por ejemplo Ethernet) es esencial para su comunicación con el exterior. Los microcontroladores, empleados sobre todo en sistemas empujados, disponen de pines de E/S de propósito general (conocidos como GPIO, de *General Purpose Input/Output*) que se conectan a diversos tipos de sensores y de actuadores.

Indistintamente del tipo de dispositivo, el procedimiento para comunicarnos con él desde un programa será siempre el mismo. En este capítulo conoceremos dicho método apoyándonos en los dispositivos de E/S que nos ofrece el simulador Ripes, y se introducirán en la parte final los detalles específicos a tener presentes cuando se opera con hardware real.

3.2 Dispositivos de E/S en Ripes

Ripes nos ofrece tres tipos diferentes de dispositivos: una **matriz de leds** (*LED Matrix*), un **panel de microinterruptores** (*Switches*) y un **conjunto de pulsadores** (*D-Pad*). De cada uno de ellos se pueden añadir al espacio de trabajo el número de copias que se desee, actuando cada una de ellas como un dispositivo distinto: a cada copia se le asigna un espacio de E/S diferente, por lo que son, en efecto, dispositivos distintos aunque del mismo tipo.

El botón I/O, en la barra lateral izquierda de Ripes, da paso al panel de dispositivos. Este se divide en tres subpaneles: el apartado *Devices* sirve para agregar al área central los dispositivos con los que queramos contar, y el subpanel derecho facilita la configuración del dispositivo seleccionado en la parte central.

3.2.1 Configuración de un dispositivo

Veamos cómo configurar un dispositivo concreto para utilizarlo desde un programa propio. Tras iniciar Ripes, reproduce los siguientes pasos:

1. Abre el panel *I/O* con el botón correspondiente. El área central aparece vacía en principio.
2. Haz doble clic sobre el elemento *LED Matrix* del apartado *Devices*. En la zona central aparece una matriz de leds con una configuración por defecto.
3. En la parte derecha se indica el alto (número de filas) y ancho (número de columnas) de la matriz. Haz clic sobre los valores y cámbialos para tener una pantalla de 64×64 leds.
4. La ventana que muestra la matriz de leds tiene un botón, debajo del de cierre, que sirve para fijar la vista del dispositivo. Haz clic sobre él, de forma que la matriz esté siempre visible aunque volvamos al editor de código.

El panel de parámetros y la ventana con la matriz resultante se muestran en las Figuras 3.1 y 3.2.

Parameters	
Name	Value
Height	64
Width	64
LED size	8

Figura 3.1: Panel de parámetros del dispositivo: se fijan el alto y el ancho a 64 para obtener una pantalla de 64×64 leds.

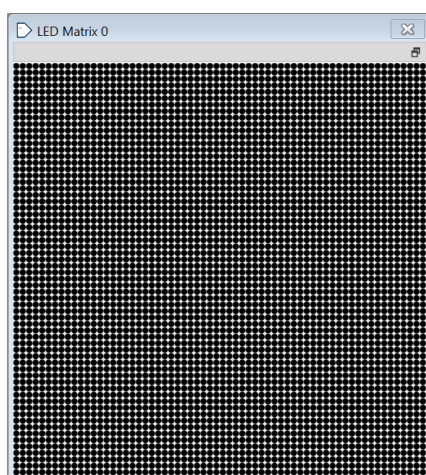


Figura 3.2: La matriz de 64×64 leds recién creada. El botón situado bajo el de cierre fija su vista para que permanezca visible al volver al editor de código.

Ripes permite crear varios dispositivos del mismo tipo, de forma que sería posible contar con varias matrices de leds para mostrar distintas informaciones: solo hay que repetir el paso 2 las veces que necesitemos. De manera análoga configuraríamos otros dispositivos, como el número de interruptores que queremos tener en un dispositivo *Switches*.

Por cada dispositivo se definirá un conjunto de **constantes**, visibles en la parte inferior derecha del panel, que llevaremos a nuestro programa para saber, por ejemplo, cuántas filas y columnas tiene nuestra matriz de leds o cuál es la dirección de memoria a partir de la cual están los valores que determinan su color. En la parte superior de ese mismo panel, Ripes describe además cómo se interpretan los valores de cada led: 24 bits tratados como tres bytes que denotan intensidad de rojo, verde y azul. Su fórmula para la posición de un led, $\text{offset} = (y + x \times \text{N_LEDS_ROW}) \times 4$, llama x a la fila y y a la columna; más adelante la escribiremos con los nombres de las constantes.

3.2.2 Eliminación de dispositivos

Los dispositivos agregados en este panel de Ripes permanecerán activos (y disponibles para el código de nuestro programa) a pesar de que cambiemos a la vista del cauce de ejecución o del editor. Que no estén visibles no implica que dejen de existir.

Dado que cada dispositivo precisa un espacio de memoria, en el que se mapean los registros de E/S que permiten comunicarse con él, cuantos más haya activos más rango de direcciones tendremos ocupadas. Es posible liberar dicho espacio usando el botón rojo en forma de aspa que hay en la esquina superior derecha de la ventana contenedora del dispositivo. Esto no solo lo hace desaparecer visualmente, sino que también elimina la definición de las constantes que representan las direcciones que tenía asociadas.

3.3 Entrada/salida mapeada

La **entrada/salida mapeada en memoria** es la técnica que hace posible todo lo que sigue, y su nombre la describe bien: los registros a través de los cuales se gobierna cada dispositivo no ocupan un espacio aparte, sino que están asociados a direcciones de memoria concretas, casi como si fueran variables. Escribir en esa dirección con **sw** envía un dato o una orden al dispositivo, y leer con **lw** consulta su estado. No hacen falta, por tanto, instrucciones especiales de E/S: una vez que sabemos en qué direcciones está mapeado cada dispositivo, para comunicarnos con él nos bastan las **instrucciones de carga y almacenamiento** que conocimos en el Capítulo 1.

Según la naturaleza de un dispositivo, en su espacio de E/S se contemplarán solo operaciones de lectura (R), solo escrituras (W) o ambas (R/W).

En la página *I/O*, Ripes define para cada dispositivo un conjunto de **constantes** (su dirección base y otros valores) que podemos usar en el programa. No es necesario copiarlas: el código las lee directamente de esa página. A continuación se presenta cada dispositivo con sus constantes, lo que significan y un ejemplo de uso. Eso sí, sus valores dependen de la sesión: Ripes asigna las direcciones base de forma consecutiva a partir de `0xf0000000`, en el orden en que se añaden los dispositivos, y las recalcula si cambia el tamaño de alguno. Los valores de las tablas siguientes corresponden a añadir, por este orden, la matriz de 64×64 leds, el panel de 12 microinterruptores y el D-Pad; en el programa hay que usar siempre el nombre de la constante, nunca el número.

3.3.1 Matriz de leds

La matriz de leds es un dispositivo de **lectura y escritura** (R/W): se escribe en ella para encender sus leds y se lee para consultar su estado. Su alto y su ancho se fijan al configurarla, como se mostró al principio del capítulo (Figuras 3.1 y 3.2). Ripes define para ella las constantes de la Tabla 3.1.

Constante	Valor	Para qué sirve
LED_MATRIX_O_BASE	0xf0000000	Dirección donde empieza el espacio de E/S del dispositivo.
LED_MATRIX_O_SIZE	0x4000	Tamaño en bytes de ese espacio (4 096 leds × 4 bytes = 16 384).
LED_MATRIX_O_WIDTH	0x40	Ancho de la matriz: 64 columnas.
LED_MATRIX_O_HEIGHT	0x40	Alto de la matriz: 64 filas.

Tabla 3.1: Constantes del dispositivo matriz de leds.

Cómo se guarda la matriz en memoria. La matriz se almacena como una sucesión de leds, uno tras otro, a partir de la dirección LED_MATRIX_O_BASE. Como cada led ocupa 4 bytes, el espacio total es LED_MATRIX_O_WIDTH × LED_MATRIX_O_HEIGHT × 4 bytes, que es justo el valor de LED_MATRIX_O_SIZE. Los leds se guardan *por filas*: de arriba abajo y, dentro de cada fila, de izquierda a derecha. Los primeros LED_MATRIX_O_WIDTH leds forman la fila superior, los siguientes la segunda, y así hasta la última. Por eso el led de la fila f y la columna c (contando ambas desde 0) se encuentra en la dirección

$$\text{LED_MATRIX_O_BASE} + (f \times \text{LED_MATRIX_O_WIDTH} + c) \times 4.$$

Sumar 4 a una dirección nos lleva al led de su derecha; sumar LED_MATRIX_O_WIDTH × 4, al que está justo debajo. El primer led (la esquina superior izquierda) está, por tanto, en LED_MATRIX_O_BASE, y el siguiente hacia la derecha en LED_MATRIX_O_BASE + 4.

Cómo se elige el color de un led. El color lo determina el valor de 4 bytes que escribimos en la dirección del led. El byte más alto no se utiliza; los otros tres son las intensidades de **rojo**, **verde** y **azul** (el modelo RGB), cada una entre 0 y 255 (de 0x00 a 0xff). El color se escribe entonces como 0xRRGGBB: los dos primeros dígitos hexadecimales son el rojo, los dos del medio el verde y los dos últimos el azul. Cuanto mayor es una componente, con más fuerza brilla ese color primario, y al mezclar las tres se consigue el resto. Poner las tres a cero, es decir escribir 0x000000, **apaga** el led. La Tabla 3.2 recoge algunos ejemplos.

Valor	Rojo	Verde	Azul	Color
0xff0000	255	0	0	rojo
0x00ff00	0	255	0	verde
0x0000ff	0	0	255	azul
0xffff00	255	255	0	amarillo
0x00ffff	0	255	255	cian
0xff00ff	255	0	255	magenta
0xffffffff	255	255	255	blanco
0x000000	0	0	0	apagado

Tabla 3.2: Ejemplos de color para un led. El valor se interpreta como 0xRRGGBB: cada par de dígitos hexadecimales es la intensidad de rojo, verde y azul, de 0x00 (nada) a 0xff (máximo).

Con esto ya sabemos encender un led del color que queramos o apagarlo. El Listado 3.1 recorre toda la matriz cambiando el color de cada led para formar un patrón: guarda en `t0` la dirección del primer led, calcula en `t1` la dirección donde termina y avanza de 4 en 4 escribiendo en cada vuelta un color distinto, hasta llegar al final.

```
1  # t0: puntero al led actual
2  # t1: direccion donde termina la matriz
3  # a0: color que se escribe (cambia en cada vuelta)
4  # a7: codigo de servicio para ecall
5  .text
6
7  li t0, LED_MATRIX_0_BASE
8  li t1, LED_MATRIX_0_SIZE
9
10 add t1, t1, t0
11 li a0, 0
12
13 bucle:
14 sw a0, 0, t0
15 addi a0, a0, 0x7ff
16 addi t0, t0, 4
17
18 bgt t1, t0, bucle
19
20 li a7, 10
21 ecall
```

Listado 3.1: Llenar la matriz de leds con diferentes colores.

En lugar de avanzar de 4 en 4 podríamos sumar `LED_MATRIX_0_WIDTH × 4` para saltar a la misma posición de la fila siguiente, lo que permite dibujar líneas, rectángulos y otras figuras con unos pocos cálculos.

3.3.2 Panel de microinterruptores

El panel de microinterruptores es un dispositivo de **solo lectura** (R). Su único parámetro configurable es el número de interruptores (por defecto 8, hasta un máximo de 32), como muestra la Figura 3.3. Todos comparten un único registro de una palabra (32 bits) en el que cada bit refleja el estado de un interruptor: 1 si está activado y 0 en caso contrario. Ripes define las constantes de la Tabla 3.3.

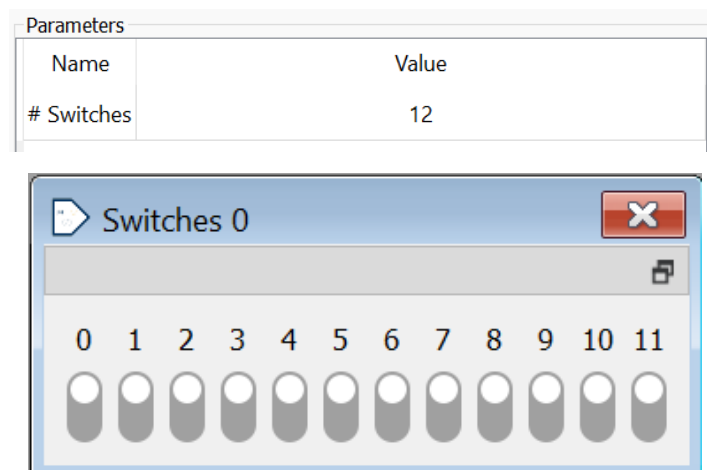


Figura 3.3: Configuración (arriba) y ventana (abajo) de un panel con 12 microinterruptores. Cada interruptor aporta un bit al registro de estado.

Constante	Valor	Para qué sirve
SWITCHES_0_BASE	0xf0004000	Dirección base del registro de estado.
SWITCHES_0_SIZE	0x4	Tamaño en bytes: una palabra. No cambia con el número de interruptores.
SWITCHES_0_N	0xc	Número de microinterruptores (12 en el panel de la Figura 3.3; 8 por defecto) y, por tanto, de bits usados en la palabra.

Tabla 3.3: Constantes del dispositivo panel de microinterruptores.

El registro reúne el estado de los doce interruptores del panel de la Figura 3.3, un bit por interruptor: el bit 0 es el interruptor 0, el bit 1 el interruptor 1, y así hasta el 11. Comprobar si uno concreto está activo es una **operación a bajo nivel**: un *AND* bit a bit entre ese registro y una máscara con un único 1 en la posición de ese interruptor. Por ejemplo, para el interruptor 5 en un panel de 12:

	0000	0010	0101	estado (interruptores 0, 2 y 5 subidos)
AND	0000	0010	0000	máscara del interruptor 5
	0000	0010	0000	distinto de 0: el interruptor 5 está subido

Si ese interruptor estuviera bajado, su bit sería 0 y el AND daría cero.

3.3.3 Conjunto de pulsadores

El conjunto de pulsadores (*D-Pad*) es también de **solo lectura** (R) y no admite configuración: siempre son cuatro pulsadores. A diferencia del panel de microinterruptores, ocupa cuatro palabras, una por pulsador (Figura 3.4). Ripes define las constantes de la Tabla 3.4.

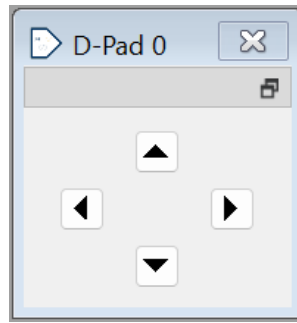


Figura 3.4: El dispositivo D-Pad, con sus cuatro pulsadores.

Constante	Valor	Para qué sirve
D_PAD_0_BASE	0xf0004004	Dirección base del dispositivo.
D_PAD_0_SIZE	0x10	Tamaño en bytes: cuatro palabras, una por pulsador.
D_PAD_0_UP	0xf0004004	Dirección del pulsador «arriba».
D_PAD_0_DOWN	0xf0004008	Dirección del pulsador «abajo».
D_PAD_0_LEFT	0xf000400c	Dirección del pulsador «izquierda».
D_PAD_0_RIGHT	0xf0004010	Dirección del pulsador «derecha».

Tabla 3.4: Constantes del dispositivo conjunto de pulsadores. Cada pulsador tiene además una constante con el sufijo `_OFFSET` que da su desplazamiento respecto a `BASE` (0x0, 0x4, 0x8 y 0xc).

Al leer la dirección de un pulsador obtenemos un valor distinto de cero si está pulsado. Cuando su ventana está en primer plano, el dispositivo responde también a las teclas W, A, S y D, como los mandos de muchos juegos. Los botones tienen cierta latencia y, además, leerlos sin pausa puede bloquear la aplicación, así que conviene esperar tras cada lectura con un breve bucle vacío. Leerlos no exige nada nuevo: cada pulsador es una carga más.

```

1 lw    t0, D_PAD_0_UP    # 1 mientras la flecha arriba esta pulsada
2 bnez  t0, pulsado       # actua en consecuencia

```

Listado 3.2: Leer el estado de un pulsador.

Con esa pieza, las esperas entre lecturas y el control de flujo del Capítulo 2 se construyen programas interactivos completos; te esperan en los ejercicios del guion de la práctica.

3.4 Otra entrada: recibir datos por la consola

Los dispositivos del panel *I/O* no son la única manera de que un programa reciba información. La consola, que hasta ahora solo usábamos para escribir, también sabe escuchar: Ripes implementa el servicio **Read** (código 63, heredado del convenio de GNU/Linux) [3], que recoge lo que se teclea en ella. La convención de registros es la habitual de `ecall`, con algún argumento más:

- `a7` = 63: el código del servicio.
- `a0` = 0: el descriptor de la consola (la entrada estándar).
- `a1`: la dirección del buffer donde dejar lo tecleado.
- `a2`: el número máximo de bytes a leer.

- A la vuelta, `a0` contiene cuántos bytes se han leído.

Al ejecutar el `ecall` el programa queda a la espera: escribe en la consola, pulsa Intro y la ejecución continúa. El Listado 3.3 lo demuestra haciendo de eco: lee una línea y la reimprime.

```

1  # a7: codigo de servicio para ecall
2  # a0: descriptor (0, la consola) y, a la vuelta, bytes leídos
3  # a1: direccion del buffer donde se deja lo tecleado
4  # a2: maximo de bytes a leer
5      .bss
6
7  buf: .word 0, 0, 0, 0      # 16 bytes a cero para lo tecleado
8
9      .text
10
11  li a7, 63                # servicio Read
12  li a0, 0                 # descriptor 0: la consola
13  la a1, buf
14  li a2, 15                # como mucho 15 bytes: queda un 0 al final
15  ecall                   # espera a que escribas y pulses Intro
16
17  la a0, buf               # eco: reimprime lo tecleado
18  li a7, 4
19  ecall
20
21  li a7, 10
22  ecall

```

Listado 3.3: Eco: leer de la consola y reimprimir lo tecleado.

Lo que llega al buffer son caracteres, es decir, códigos ASCII como los de la Sección 1.4.4: si tecleas 25 y pulsas Intro, recibes los bytes de «2», de «5» y del salto de línea (0x32, 0x35 y 0x0A), y `a0` vale 3; no el número 25.

3.5 E/S en hardware físico

El concepto de la E/S mapeada en memoria, mediante la cual podemos comunicarnos con dispositivos externos sin más que leer o escribir de ciertas posiciones de memoria (con instrucciones de carga y almacenamiento que nos son familiares), es extensible al hardware físico con CPU RISC-V. Al igual que hemos hecho con los dispositivos que ofrece el simulador Ripes, el primer paso es conocer cuáles son las direcciones asignadas a los puertos de comunicación y cuál es la finalidad que tienen.

Hemos asumido desde un principio que podemos acceder a esas posiciones de memoria sin ningún obstáculo, al igual que hacemos con las direcciones que corresponden a nuestro segmento de datos. Esto, sin embargo, no siempre es cierto. En hardware real, los dispositivos son recursos que suelen ser compartidos por todas las aplicaciones que haya en ejecución, de ahí que sea necesario controlar el acceso a los mismos para evitar conflictos.

3.5.1 E/S en microcontroladores

Un microcontrolador (MCU, *Microcontroller Unit*) es un tipo de circuito integrado que aúna una CPU, cierta cantidad de memoria RAM y la circuitería necesaria para gestionar dispositivos de

E/S de forma bastante directa, sin precisar otros componentes de apoyo o intermediarios. Sus recursos computacionales suelen ser menores que los de un microprocesador estándar (cuentan con menos núcleos, operan a menor velocidad y no son tan sofisticados) pero, a cambio, requieren mucha menos energía, disipan poco calor y apenas demandan refrigeración, lo que les hace muy interesantes para su uso en sistemas empotrados y de tipo IoT.

Los MCU son sencillos y, por esa razón, solo suelen contar con el modo de funcionamiento esencial: el **modo máquina**. Esto implica que el código puede acceder a todos los recursos sin ninguna limitación y, al tiempo, sin protección de fallos por usos indebidos. Es un contexto en el que nuestro código no tendría limitaciones para leer o escribir de cualquier dirección de memoria, de forma similar a como hemos hecho en el simulador Ripes.

Por regla general, los dispositivos de E/S controlados por un MCU estarán vinculados a los pines GPIO. Estos son configurables, a través de una matriz de interconexiones, de forma que un pin puede ser de entrada, de salida o ambos, según lo que conectemos al mismo. Los pasos para operar con GPIO son los siguientes:

1. Seleccionar si el pin será de lectura o de escritura. Para ello se escribe la configuración en un registro de control.
2. Configurar el pin según el modo de funcionamiento. Para entrada es posible activar un filtro, mientras que la salida determina cuándo está activa la señal.
3. Leer o escribir el valor del pin en su registro de datos (de entrada o de salida), una vez configurado en los dos pasos previos.

3.5.2 E/S en ordenadores

En este apartado interpretaremos el sustantivo *ordenador* como un equipo en el que se instala un sistema operativo estándar, por ejemplo GNU/Linux, y es él quien se encarga de todos los aspectos de bajo nivel, como la comunicación con los dispositivos de E/S. Ese sistema se encarga, además, de configurar el procesador de manera que las aplicaciones de usuario se ejecutan en el modo con menos privilegios, no teniendo acceso al modo máquina o supervisor. Esto implica que no se tiene acceso a cualquier dirección de memoria, aparte de no poder ejecutar ciertas instrucciones.

Según el sistema operativo que ejecute el ordenador, el procedimiento para acceder a los dispositivos de E/S será uno u otro. En GNU/Linux, como en otros derivados de Unix, casi todos los dispositivos aparecen en el sistema de archivos, en concreto en la carpeta `/dev` (las interfaces de red son la excepción más conocida): unidades de almacenamiento interno, puertos serie, dispositivos de vídeo, etc. Incluso la memoria del ordenador está ahí, en el dispositivo `/dev/mem`, lo cual permite abrirlo y leer o escribir de la memoria como si de un archivo se tratase. En caso de que el ordenador cuente con GPIO, lo cual es habitual en ordenadores de tipo SBC, encontraremos en el directorio `/dev` uno o más dispositivos con el nombre `gpiochipN`.

El procedimiento para operar sobre los GPIO desde un programa consta de varios pasos: apertura del dispositivo `/dev/gpiochip` que corresponda; uso de la llamada al sistema `IOCTL` (*Input/Output Control*) para recuperar información sobre el estado de los pines (por ejemplo si están en uso o libres, cómo están configurados, etc.), y lectura y/o escritura en los puertos asociados a los pines mediante la misma llamada `IOCTL`. El uso de `IOCTL` conlleva el uso de estructuras de datos de relativa complejidad, por lo que no resulta cómodo hacerlo directamente desde ensamblador.

Con esto cerramos el recorrido de estas prácticas. Hemos ido desde escribir las primeras instrucciones y mover datos entre registros y memoria, hasta controlar el flujo con saltos, bucles y funciones apoyadas en la pila, para terminar abriendo el procesador al exterior con la entrada/salida.

La idea que vertebra este último capítulo, comunicarse con un dispositivo leyendo y escribiendo en unas direcciones de memoria, es la misma en el simulador, en un microcontrolador o en un ordenador completo; lo que cambia de un caso a otro es quién controla el acceso a esas direcciones. El Anexo [A](#) recoge, como referencia rápida, todo el repertorio de instrucciones que hemos ido empleando.

Repertorio de instrucciones RV32IM

Este anexo recopila, como referencia rápida, el conjunto de instrucciones de la ISA RV32IM presentado en los capítulos 1 y 2 (véase la Tabla 1.2), junto con las seudoinstrucciones de uso común y su equivalencia en instrucciones reales, tal y como las expande el ensamblador (puedes comprobarlo en el panel derecho de la vista *Editor* de Ripes). Para no repetir su significado en cada fila, los parámetros que aparecen en las instrucciones son siempre los siguientes:

Parámetro	Significado
rd	Registro de destino : donde se deja el resultado
rs1, rs2, rs	Registros de origen : contienen los operandos
imm	Valor inmediato : constante incluida en la propia instrucción
desp	Desplazamiento que se suma a la dirección base
rbase	Registro que contiene la dirección base de memoria
etiqueta	Nombre dado a un punto del código o a un dato en memoria

En las comparaciones y saltos, *con signo* / *sin signo* indica cómo se interpretan los valores de los registros al compararlos. Las instrucciones señaladas con este sombreado son las de uso más habitual en los ejercicios de las Prácticas 1, 2 y 3.

Instrucción	Qué hace
Aritméticas	
add rd, rs1, rs2	rd = rs1 + rs2.
addi rd, rs1, imm	rd = rs1 + imm.
sub rd, rs1, rs2	rd = rs1 − rs2.
mul rd, rs1, rs2	rd = rs1 × rs2.
div rd, rs1, rs2	rd = cociente de la división entera rs1/rs2.
rem rd, rs1, rs2	rd = resto de la división entera rs1/rs2.
lui rd, imm	Carga los 20 bits de imm en los bits 31..12 de rd; los 12 de menor peso quedan a 0.
auipc rd, imm	rd = pc + (imm ≪ 12): versión relativa al pc de lui.
Lógicas (bit a bit)	
and rd, rs1, rs2	AND bit a bit de rs1 y rs2.
andi rd, rs1, imm	AND bit a bit de rs1 e imm.
or rd, rs1, rs2	OR bit a bit de rs1 y rs2.
ori rd, rs1, imm	OR bit a bit de rs1 e imm.
xor rd, rs1, rs2	XOR bit a bit de rs1 y rs2.
xori rd, rs1, imm	XOR bit a bit de rs1 e imm.
Desplazamiento de bits	
sll rd, rs1, rs2	Desplaza rs1 a la izquierda tantos bits como indique rs2.
slli rd, rs1, imm	Desplaza rs1 a la izquierda imm bits.
srl rd, rs1, rs2	Desplaza rs1 a la derecha (lógico) tantos bits como indique rs2.

Instrucción	Qué hace
<code>srli rd, rs1, imm</code>	Desplaza <code>rs1</code> a la derecha (lógico) <code>imm</code> bits.
<code>sra rd, rs1, rs2</code>	Desplaza <code>rs1</code> a la derecha conservando el bit de signo (aritmético).
<code>srai rd, rs1, imm</code>	Como la anterior, con la cantidad en <code>imm</code> .
Comparación	
<code>slt rd, rs1, rs2</code>	<code>rd = 1</code> si <code>rs1 < rs2</code> (con signo); 0 en caso contrario.
<code>sltu rd, rs1, rs2</code>	Como la anterior, sin signo.
<code>slti rd, rs1, imm</code>	<code>rd = 1</code> si <code>rs1 < imm</code> (con signo); 0 en caso contrario.
<code>sltiu rd, rs1, imm</code>	Como la anterior, sin signo.
Carga desde memoria	
<code>lw rd, desp(rbase)</code>	Carga en <code>rd</code> la palabra (32 bits) de la dirección <code>rbase + desp</code> .
<code>lh rd, desp(rbase)</code>	Carga media palabra (16 bits) con extensión de signo.
<code>lhu rd, desp(rbase)</code>	Carga media palabra (16 bits) sin signo.
<code>lb rd, desp(rbase)</code>	Carga un byte (8 bits) con extensión de signo.
<code>lbu rd, desp(rbase)</code>	Carga un byte (8 bits) sin signo.
<code>ld rd, desp(rbase)</code>	Carga una doble palabra (64 bits); solo en RV64.
<code>ldu rd, desp(rbase)</code>	Carga una palabra (32 bits) sin signo; solo en RV64.
Almacenamiento en memoria	
<code>sw rs2, desp(rbase)</code>	Escribe la palabra (32 bits) de <code>rs2</code> en la dirección <code>rbase + desp</code> .
<code>sh rs2, desp(rbase)</code>	Escribe los 16 bits de menor peso de <code>rs2</code> .
<code>sb rs2, desp(rbase)</code>	Escribe el byte de menor peso de <code>rs2</code> .
<code>sd rs2, desp(rbase)</code>	Escribe una doble palabra (64 bits); solo en RV64.
Salto	
<code>beq rs1, rs2, etiqueta</code>	Salta si <code>rs1 = rs2</code> .
<code>bne rs1, rs2, etiqueta</code>	Salta si <code>rs1 ≠ rs2</code> .
<code>bge rs1, rs2, etiqueta</code>	Salta si <code>rs1 ≥ rs2</code> (con signo).
<code>bgeu rs1, rs2, etiqueta</code>	Salta si <code>rs1 ≥ rs2</code> (sin signo).
<code>blt rs1, rs2, etiqueta</code>	Salta si <code>rs1 < rs2</code> (con signo).
<code>bltu rs1, rs2, etiqueta</code>	Salta si <code>rs1 < rs2</code> (sin signo).
<code>jal rd, etiqueta</code>	Guarda en <code>rd</code> la dirección de la instrucción siguiente (<code>pc + 4</code>) y salta a <code>etiqueta</code> .
<code>jalr rd, rs1, imm</code>	Guarda <code>pc + 4</code> en <code>rd</code> y salta a la dirección <code>rs1 + imm</code> .
Sistema	
<code>ecall</code>	Invoca al servicio del sistema cuyo código está en <code>a7</code> (véase <i>Help→System calls</i> en Ripes).
<code>ebreak</code>	Detiene la ejecución para ceder el control al depurador. Ripes 2.2.6 no la implementa: su ensamblador la rechaza.
Seudoinstrucciones de uso común	

Instrucción	Qué hace
<code>li rd, imm</code>	<code>rd = imm.</code> Equivale a: <code>addi rd, zero, imm</code> ; si el valor no cabe en 12 bits, <code>lui + addi</code> : la <code>addi</code> lleva los 12 bits bajos leídos con signo y la <code>lui</code> los 20 altos, más 1 si el bit 11 del valor es 1 (<code>li t0, 0x12345fff</code> → <code>lui t0, 0x12346 + addi t0, t0, -1</code>); si los 12 bits bajos son 0, basta la <code>lui</code> .
<code>la rd, etiqueta</code>	Carga en <code>rd</code> la <i>dirección</i> de <code>etiqueta</code> . Equivale a: <code>auipc rd, %pcrel_hi(etiqueta) + addi rd, rd, %pcrel_lo(etiqueta)</code> : las partes alta y baja de la distancia entre el <code>pc</code> y la <code>etiqueta</code> .
<code>lw rd, etiqueta</code>	Carga en <code>rd</code> el <i>dato</i> almacenado en <code>etiqueta</code> . Equivale a: <code>auipc + lw</code> (dos instrucciones reales).
<code>mv rd, rs</code>	Copia <code>rs</code> en <code>rd</code> . Equivale a: <code>addi rd, rs, 0</code>
<code>not rd, rs</code>	Complemento a uno de <code>rs</code> (invierte todos los bits). Equivale a: <code>xori rd, rs, -1</code>
<code>neg rd, rs</code>	Complemento a dos de <code>rs</code> (cambia el signo). Equivale a: <code>sub rd, zero, rs</code>
<code>nop</code>	No realiza ningún trabajo útil; consume ciclos de reloj. Equivale a: <code>addi zero, zero, 0</code>
<code>j etiqueta</code>	Salto incondicional sin guardar dirección de retorno. Equivale a: <code>jal zero, etiqueta</code>
<code>call etiqueta</code>	Llama a la subrutina <code>etiqueta</code> guardando la dirección de retorno en <code>ra</code> . Equivale a: <code>auipc ra, %pcrel_hi(etiqueta) + jalr ra, ra, %pcrel_lo(etiqueta)</code> ; Ripes usa siempre estas dos instrucciones, esté cerca o lejos la <code>etiqueta</code> .
<code>ret</code>	Vuelve a la dirección de retorno almacenada en <code>ra</code> . Equivale a: <code>jalr zero, ra, 0</code>
<code>bgt rs1, rs2, etiqueta</code>	Salta si <code>rs1 > rs2</code> (con signo). Equivale a: <code>blt rs2, rs1, etiqueta</code> ; <code>bgtu</code> (sin signo) → <code>bltu rs2, rs1</code> .
<code>ble rs1, rs2, etiqueta</code>	Salta si <code>rs1 ≤ rs2</code> (con signo). Equivale a: <code>bge rs2, rs1, etiqueta</code> ; <code>bleu</code> (sin signo) → <code>bgeu rs2, rs1</code> .
<code>beqz rs1, etiqueta</code>	Salta si <code>rs1 = 0</code> . Equivale a: <code>beq rs1, zero, etiqueta</code>
<code>bnez rs1, etiqueta</code>	Salta si <code>rs1 ≠ 0</code> . Equivale a: <code>bne rs1, zero, etiqueta</code>
<code>bgez rs1, etiqueta</code>	Salta si <code>rs1 ≥ 0</code> . Equivale a: <code>bge rs1, zero, etiqueta</code>
<code>bgtz rs1, etiqueta</code>	Salta si <code>rs1 > 0</code> . Equivale a: <code>blt zero, rs1, etiqueta</code>
<code>blez rs1, etiqueta</code>	Salta si <code>rs1 ≤ 0</code> . Equivale a: <code>bge zero, rs1, etiqueta</code>

Operadores sobre direcciones (no son instrucciones)

Los resuelve el ensamblador al traducir el programa: descomponen la dirección de una `etiqueta` en sus dos partes, como vimos en la Sección 1.6.

Instrucción	Qué hace
<code>%hi(etiqueta)</code>	Los 20 bits de mayor peso de la dirección de <code>etiqueta</code> ; se usa con <code>lui</code> : <code>lui t0, %hi(par)</code> .
<code>%lo(etiqueta)</code>	Los 12 bits de menor peso de la dirección; completa a la anterior con <code>addi</code> : <code>addi t0, t0, %lo(par)</code> . En Ripes solo ensambla si el bit 11 de la dirección vale 0; si no, usa <code>la</code> .
<code>%pcrel_hi(etiqueta)</code>	Como <code>%hi</code> , pero con la dirección relativa al <code>pc</code> ; se usa con <code>auipc</code> .
<code>%pcrel_lo(etiqueta)</code>	Como <code>%lo</code> , para completar la dirección relativa iniciada con <code>auipc</code> ; se calcula respecto a la dirección de la <code>auipc</code> (véase la Sección 1.6).

Referencias

- [1] Francisco Charte Ojeda. *Programación en ensamblador RISC-V*. Torre de Babel, 2025. ISBN 978-84-09-76297-2. Universidad de Jaén, Colección de Recursos Educativos Abiertos (CREA). Disponible en: <https://hdl.handle.net/10953.1/26479>.
- [2] David A. Patterson y John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. Morgan Kaufmann (Elsevier), 2018. ISBN 978-0-12-812275-4.
- [3] Morten B. Petersen. Ripes: A graphical processor simulator and assembly editor for the RISC-V ISA. Repositorio oficial del proyecto en GitHub. Disponible en: <https://github.com/mortbopet/Ripes>.